A Synopsis of Project on

LUMIS : LLM Based Unified Multimodal Intelligent System

Submitted in partial fulfillment of the requirements for the award
of the degree of

Bachelor of Engineering

in

Computer Science and Engineering (Data Science)

by

Dalbirsingh Matharu(21107005)
Umesh Pawar(21107014)
Shreyas Patil(21107061)
Vedant Parulekar(21107034)

Under the Guidance of

Ms. Sarala Mary
Ms. Richa Singh

Department of Computer Science and Engineering (Data Science)
A.P. Shah Institute of Technology
G.B.Road,Kasarvadavli, Thane(W)-400615
UNIVERSITY OF MUMBAI

Academic Year 2024-2025

Approval Sheet

This Project Synopsis Report entitled ***"LUMIS : LLM Based Unified Multimodal Intelligent System"*** Submitted by ***"Dalbirsingh Matharu"(21107005),"Umesh Pawar"(21107014),"Shreyas Patil"(21107061),"Vedant Parulekar"(21107034)***is approved for the partial fulfillment of the requirenment for the award of the degree of ***Bachelor of Engineering*** in ***Compurt Science and Engineering (Data Science)*** from ***University of Mumbai***.

(Ms. Richa Singh)
Co-Guide

(Ms. Sarala Mary)
Guide

Ms. Anagha Aher
HOD,CSE (Data Science)

Place:A.P.Shah Institute of Technology, Thane
Date:

CERTIFICATE

This is to certify that the project entitled ***"LUMIS : LLM Based Unified Multimodal Intillegent System"*** submitted by ***"Dalbirsingh Matharu" (21107005),"Umesh Pawar" (21107014),"Shreyas Patil" (21107061),"Vedant Parulkear" (21107034)*** for the partial fulfillment of the requirement for award of a degree ***Bachelor of Engineering*** in ***Computer Science and Engineering (Data Science)***,to the University of Mumbai,is a bonafide work carried out during academic year 2024-2025.

(Ms. Richa Singh)
Co-Guide

(Ms. Sarala Mary)
Guide

Ms. Anagha Aher
HOD,CSE (Data Science)

Dr. Uttam D.Kolekar
Principal

External Examiner(s)

1.

2.

Internal Examiner(s)

1.

2.

Place:A.P.Shah Institute of Technology, Thane
Date:

Acknowledgement

We have great pleasure in presenting the synopsis report on **LUMIS : LLM Based Unified Multimodal Intelligent System** We take this opportunity to express our sincere thanks towards our guide **Ms.Sarala Mary** & Co-Guide **Ms.Richa Singh** for providing the technical guidelines and suggestions regarding line of work. We would like to express our gratitude towards his constant encouragement, support and guidance through the development of project.

We thank **Ms. Anagha Aher** Head of Department for her encouragement during the progress meeting and for providing guidelines to write this report.

We express our gratitude towards BE project co-ordinator **Ms. Poonam Pangarkar**, for being encouraging throughout the course and for their guidance.

We also thank the entire staff of APSIT for their invaluable help rendered during the course of this work. We wish to express our deep gratitude towards all our colleagues of APSIT for their encouragement.

Dalbirsingh Matharu
(21107005)

Umesh Pawar
(21107014)

Shreyas Patil
(21107061)

Vedant Parulekar
(21107034)

Declaration

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, We have adequately cited and referenced the original sources. We also declare that We have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission. We understand that any violation of the above will be cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Dalbirsingh Matharu(21107005)

Umesh Pawar(21107014)

Shreyas Patil(21107061)

Vedant Parulekar(21107034)

Date:

Abstract

Project Lumis is an innovative initiative focused on developing a unified multimodal intelligent system powered by advanced large language models (LLMs). It aims to integrate a wide range of capabilities, including text generation from various inputs and images, text summarization, YouTube video transcription, website transcription, image-to-text conversion, and multiformat file retrieval through APIs such as OpenAI and Gemini.
At the core of LUMIS is a suite of intelligent AI agents designed with the LangChain framework. This allows the system to function as an adaptive assistant that learns from user interactions, makes informed decisions, and automates complex tasks. Emphasizing user-friendliness and voice assistance, Lumis provides a versatile solution for diverse multimodal tasks.
Key features include an intelligent code assistant that offers real-time code comprehension, error detection, and suggestion generation, seamlessly integrating with popular code editors to enhance developer productivity. Additionally, LUMIS features SQL integration for streamlined database interactions and an Air Canva functionality for creating visually appealing content effortlessly. Continuous feature updates will ensure that LUMIS remains at the forefront of technological advancements, effectively meeting the evolving needs of its users.

Keywords: LLM, multimodal, text generation, text summarization, video transcription, website transcription, image-to-text, file aggregation, AI agents, LangChain, Gemini, OpenAI, intelligent assistant, automation, adaptive learning, user-friendly, voice assistant, code assistant, SQL integration, Air Canva.

Contents

1 Introduction **1**
- 1.1 Motivation . . . 2
- 1.2 Problem Statement . . . 3
- 1.3 Objectives . . . 4
- 1.4 Scope . . . 4

2 Literature Review **6**
- 2.1 Comparative Analysis of Recent Study . . . 6

3 Project Design **12**
- 3.1 Existing System . . . 12
- 3.2 Proposed System Architecture . . . 14
- 3.3 UML Diagrams . . . 16
  - 3.3.1 Activity Diagram . . . 16
  - 3.3.2 Use Case Diagram . . . 18
  - 3.3.3 Sequence Diagram . . . 19

4 Project Implementation **24**
- 4.1 Code Snippets . . . 24
- 4.2 Steps to access the System . . . 27
- 4.3 Timeline Sem VIII . . . 31

5 Testing **33**
- 5.1 Software Testing . . . 33
- 5.2 Functional Testing . . . 35

6 Result and Discussions **40**

7 Conclusion **43**

8 Future Scope **45**

Appendices **49**
- Appendix-B . . . 49

Publication **51**

List of Figures

3.1 A generalist multimodal architecture pipeline 12
3.2 A generalist multimodal architecture pipeline 13
3.3 Proposed System Architecture of Lumis 14
3.4 Activity Diagram of Lumis 17
3.5 Use Case Diagram of Lumis 18
3.6 Sequence Diagram of Image and Text Processing Module 19
3.7 Sequence Diagram of T.A.P.A.S Module 20
3.8 Sequence Diagram of S.Q.L.A.G.E Module 21
3.9 Sequence Diagram of PDF RAG Module 22
3.10 Sequence Diagram of YT Transcription Module 23

4.1 PDF Summarization Function 25
4.2 Image Processing & Text Analysis Function 25
4.3 YouTube Video Transcript Summarization Function 26
4.4 Summarize Multiple PDFs 27
4.5 Chatbot 28
4.6 SQLAGE 28
4.7 YT Transcription 29
4.8 T.A.P.A.S 30
4.9 Timeline of Project 32

6.1 Time Series Analysis of Task Execution Time vs Complexity 40
6.2 Execution Time vs Complexity in Chatbot System 41
6.3 Execution Time vs Query Complexity in Chatbot System 42

List of Tables

2.1 Comparative Analysis of Literature Survey 9

5.1 Functional Testing - Code and Core Interaction Modules 36
5.2 Functional Testing - Retrieval and Data Processing Modules 37

List of Abbreviations

LUMIS:	LLM Based Unified Multimodal Intelligent System
SQLAGE:	SQL Automated Generation and Execution
TAPAS:	Technical Assistance Platform for Advanced Solution
SQL:	Structured Query Language
RAG:	Retrieval Augmented Generation
LLM:	Large Language Model
API:	Application Programming Interface
AI:	Artificial Intelligence
GPT:	Generative Pre-trained Transformer
OCR:	Optical Character Recognition
NLP:	Natural Language Processing
YT:	YouTube

Chapter 1

Introduction

LUMIS is an innovative initiative aimed at transforming the landscape of software development through the application of advanced large language models (LLMs). As software development grows increasingly complex, developers face a myriad of challenges, including the integration of various tools, the need for quick error resolution, and the management of large volumes of information. To address these challenges, there is a pressing need for a comprehensive solution that integrates multiple functionalities into a cohesive platform. **LUMIS** responds to this need by enhancing developer productivity and streamlining workflows, ultimately facilitating a more efficient development process. Central to **LUMIS** is the integration of diverse functionalities specifically designed to meet the evolving needs of developers and content creators alike. The system features robust text generation capabilities, enabling users to produce high-quality written content with ease. Additionally, **LUMIS** incorporates transcription services that convert audio and video materials into editable text formats, ensuring that users can efficiently manage and utilize their information. The platform also offers image-to-text conversion, allowing users to extract and manipulate data from visual content effectively. By leveraging APIs from industry leaders such as OpenAI and Gemini, **LUMIS** ensures that users benefit from state-of-the-art technology, significantly improving the overall effectiveness and versatility of the system. One of the standout features of **LUMIS** is its real-time code assistant, which provides immediate feedback on coding errors as they occur. This capability not only reduces the time developers spend debugging but also allows them to maintain focus on their core tasks, thereby enhancing productivity. Furthermore, the integration of SQL functionalities streamlines database management by automating query generation and extraction, enabling developers to efficiently handle large datasets without unnecessary complexity. This feature significantly empowers users, allowing them to execute data-related tasks with confidence and precision.

Committed to continuous improvement, **LUMIS** evolves alongside user needs through regular updates that introduce new features and enhancements. This proactive approach ensures that the platform remains relevant and effective in a rapidly changing technological landscape. Moreover, **LUMIS** is designed with a user-friendly interface that promotes accessibility for developers of all skill levels, fostering an intuitive experience that minimizes the learning curve often associated with advanced tools.

In summary, **LUMIS** stands out as a comprehensive platform that integrates advanced functionalities—text generation, transcription, image-to-text conversion, real-time code assistance, and SQL management—to address the multifaceted challenges of modern application development. By focusing on usability and prioritizing continuous updates, **LUMIS** enhances productivity and streamlines workflows, making it an essential resource for developers seeking to navigate the complexities of today's technology landscape.

1.1 Motivation

Project Lumis is motivated by the clear absence of a fully integrated large language model (LLM) solution that provides a comprehensive suite of functionalities for developers and content creators. In today's fast-paced software development landscape, developers often rely on multiple, disjointed tools for various tasks, including code assistance, SQL management, text generation, and content creation. This fragmentation results in inefficiencies, disrupted workflows, and increased complexity, as developers must switch between different systems to complete their projects. Such an environment can hinder productivity, slow down development processes, and complicate the overall management of tasks.

Lumis addresses this challenge by offering a unified platform that consolidates essential features into one seamless environment. With advanced automated SQL generation and extraction, developers can manage databases with ease, streamlining data retrieval and manipulation processes. Furthermore, Lumis incorporates text generation and summarization capabilities, allowing users to create and refine written content quickly and efficiently. By leveraging the Gemini API and the LangChain framework, Lumis provides powerful tools for YouTube video transcription and website transcription, enabling users to convert audio and web content into editable text formats. The platform also supports multiformat file retrieval and aggregation, making it easier for users to manage diverse sources of information in one place.

A standout feature of Lumis is its innovative code assistance powered by the TAPAS model. This feature transforms the debugging experience by allowing developers to pinpoint errors on their screens using their mouse, verbally describe the issue, and receive immediate, context-aware solutions. This interactive troubleshooting method significantly reduces the time spent on debugging, enhancing overall efficiency in the development process. Additionally, the platform includes Air Canva, a tool similar to Canva, which empowers users to create visually appealing content effortlessly.

In summary, Lumis is designed to be a comprehensive, intelligent platform that integrates critical development and content creation tools—SQL management, text generation, summarization, transcription services, and advanced code assistance—into one unified system. By focusing on continuous updates and providing a user-friendly interface, Lumis ensures that developers and content creators can work more effectively and efficiently, adapting to the increasingly complex demands of their projects. This holistic approach not only simplifies workflows but also enhances productivity, making Lumis a vital resource for anyone looking to thrive in the modern software development landscape.

1.2 Problem Statement

The software development ecosystem currently lacks a comprehensive unified platform that seamlessly integrates essential functionalities, such as real-time code assistance, text generation, content creation, and efficient workflow management. Developers often struggle with a fragmented array of tools that do not provide timely feedback or adapt to the evolving needs of modern software development, highlighting the urgent necessity for a holistic solution that encompasses all these features.

1. **Unified Platform for All Features** : Developers frequently face challenges due to the reliance on multiple disconnected tools for different tasks, including coding, SQL management, and content creation. This fragmentation leads to inefficiencies as they must transition between various platforms to complete their work, resulting in wasted time and increased cognitive strain. A cohesive platform that consolidates crucial features—such as text generation, retrieval-augmented generation (RAG), SQL capabilities, and transcription services—into one unified environment is essential. Such integration would enable developers to manage all aspects of their workflow seamlessly, enhancing overall efficiency and productivity.

2. **Real-Time Code Assistant Utilizing Mouse Pinpointing and User Audio** : Existing code assistance tools often require manual input and lack the contextual awareness needed for effective debugging. A real-time code assistant that leverages mouse pinpointing allows developers to visually identify errors on their screens while describing issues verbally. This innovative approach not only improves the debugging experience by delivering immediate, context-aware feedback but also significantly reduces the time spent on troubleshooting. By combining visual cues with audio input, this solution provides a more interactive and efficient means for developers to manage code errors, ultimately boosting productivity.

3. **Engaging Feature Using Air Canva and Automated SQL Generation and Extraction** : Current content creation tools can be complex and often lack the flexibility needed for dynamic design tasks. Integrating an engaging feature like Air Canva into the platform enables users to create visually compelling content with ease. This functionality empowers both developers and content creators to design graphics and presentations without requiring advanced design skills. Additionally, the inclusion of automated SQL generation and extraction simplifies database interactions, allowing users to generate and retrieve queries effortlessly. By making the content creation process more intuitive and enjoyable, users can produce high-quality visuals while managing their data effectively, thereby streamlining the overall development workflow.

4. **Dynamic and User-Friendly Interface** : Many existing development tools are hindered by complex and unintuitive interfaces, which can impede user adoption and reduce productivity, particularly for less experienced developers. A dynamic, user-friendly interface is crucial for making the platform accessible and easy to navigate. Such an interface would cater to users of varying skill levels and improve the overall user experience, allowing developers to concentrate on their tasks rather than struggling with convoluted tools. By emphasizing usability and interactivity, the platform can cultivate a more efficient and satisfying development environment.

1.3 Objectives

The objectives of the **LUMIS** project outline a comprehensive approach to enhancing coding and application development through the integration of advanced technologies. By focusing on real-time error detection, multimodal functionalities, automated SQL processes, and seamless technological integration, **LUMIS** aims to create a unified platform that significantly improves productivity and streamlines workflows.

1. Real-Time Code Error Detection and Resolution
LUMIS aims to develop a multimodal system that detects and resolves code errors in real time using video and audio inputs. Users can highlight errors on their screens with mouse pinpointing while describing issues through audio, ensuring immediate feedback. This dual input method reduces debugging time and minimizes disruptions, ultimately enhancing productivity.

2. Integration of Advanced Functionalities
Another key objective is to integrate functionalities like text generation, retrieval-augmented generation (RAG), and transcription into a cohesive platform. Text generation enables users to create high-quality written content, while RAG enhances information retrieval for informed decision-making. Transcription services convert audio and video content into editable text, streamlining information management and providing a powerful toolset for users.

3. Automated SQL Generation and Data Extraction
LUMIS will facilitate automated SQL generation and data extraction, improving database management efficiency. Leveraging Generative AI (GenAI), the system can generate SQL queries based on user needs, saving time and minimizing errors. Integration with OpenCV and Air Canvas technology allows for real-time problem-solving based on user-drawn annotations, enriching the user experience and simplifying data management.

4. Seamless Integration and Continuous Updates
LUMIS is committed to seamless integration of technologies and continuous updates to maintain functionality and relevance. This ensures the platform adapts to evolving user needs and technological advancements. Regular updates enhance usability and keep **LUMIS** user-friendly, providing a reliable and efficient tool that evolves alongside user requirements.

1.4 Scope

The scope of the **LUMIS** project outlines the key functionalities and features that will be developed to create a unified multimodal intelligent system. By focusing on integrated error detection, interactive debugging tools, unified AI capabilities, and an advanced user interface, **LUMIS** aims to deliver an efficient and productive experience for users.

1. Integrated Error Detection
LUMIS will develop a system that combines video, audio, and text inputs for the instant identification and resolution of coding errors. This multimodal approach ensures that developers receive immediate, context-aware feedback, significantly enhancing the efficiency of error detection and correction.

2. Interactive Debugging Tools
The project will create functionalities that enable users to actively pinpoint and correct errors on-screen. With features that provide immediate feedback, developers can interactively engage with their code, facilitating a more efficient debugging process and reducing time spent on troubleshooting.

3. Unified AI Features
LUMIS aims to merge advanced AI capabilities into a single, efficient platform. This includes text generation for creating high-quality written content, retrieval-augmented generation (RAG) for enhanced information retrieval, and transcription services for converting audio and video into editable text. The integration of these functionalities will provide users with a comprehensive toolset that streamlines their workflows.

4. Advanced User Interface and Automation
The project will focus on designing a dynamic, user-centric interface that enhances the overall user experience. Additionally, **LUMIS** will automate SQL generation and data management processes to facilitate seamless interaction with databases, ultimately improving productivity and simplifying complex tasks for users.

Chapter 2

Literature Review

The rise of artificial intelligence (AI) has brought transformative advancements across numerous sectors, with significant potential to revolutionize multimodal interactions and automate complex tasks. Large language models (LLMs), such as OpenAI and Gemini, have been at the forefront of these advancements, providing unprecedented capabilities in understanding and processing different forms of data. These models are now being integrated into unified systems capable of handling a wide array of inputs, including text, images, and videos, to deliver a more streamlined user experience. Traditional approaches to multimodal processing typically involve separate tools for each type of input, which often results in fragmented workflows, inefficiencies, and increased complexity for users. By unifying these functionalities into one cohesive platform, advanced systems aim to address these challenges, providing a seamless experience that simplifies task execution and minimizes errors. This evolution represents a significant shift in how multimodal data is processed and managed, with applications in various fields such as content creation, data analysis, and software development. This literature review explores the application of advanced technologies in developing intelligent, multimodal systems that can perform a wide range of tasks. It will also examine the challenges associated with integrating these technologies into user-centric AI platforms, including the complexities of ensuring real-time performance, adaptability to diverse user needs, and the ethical considerations of AI-driven automation. Furthermore, the review will highlight the opportunities these systems present in driving innovation and filling gaps in current research, offering insights into how such technologies can be optimized for broader and more efficient use.

2.1 Comparative Analysis of Recent Study

In recent years, the rapid evolution of artificial intelligence (AI) has led to significant progress in the development and application of Large Language Models (LLMs), Vision-Language Models (VLMs), and AI-assisted tools for code understanding. These advancements have enabled machines to perform increasingly complex tasks such as natural language generation, multimodal reasoning, and automated code comprehension with greater accuracy and fluency. As these technologies continue to grow in sophistication and scale, their adoption across academic, industrial, and real-world settings has accelerated. However, despite the notable achievements in performance and versatility, critical challenges persist—ranging from computational inefficiencies and interpretability issues to ethical risks such as misinformation, bias, and a lack of transparency.

To better understand the current landscape, it is essential to examine the methodologies, capabilities, and limitations of recent scholarly works in these domains. This comparative literature review draws on studies summarized in Table 2.1, which collectively reflect the latest research trends in retrieval-augmented generation, responsible AI design, vision-language integration, and generative tools for code analysis. The goal is to provide a structured evaluation of each study's methodological approach, innovations, and drawbacks, offering insights into both the strengths and the unresolved issues in existing models. By doing so, this review not only captures the state-of-the-art in AI model development but also underscores the urgent need for improved scalability, ethical considerations, and domain-specific adaptations in future research and applications.

1. The study by Patrick Lewis et al. (2020) [5] introduces a retrieval-augmented generation (RAG) model that enhances language generation by integrating sequence-to-sequence learning with Dense Passage Retrieval. The system uses RAG-Token and RAG-Sequence mechanisms to dynamically fetch and incorporate external information during text generation. While this approach significantly reduces hallucinations and increases factual grounding, it comes with drawbacks such as high computational overhead, dependency on static knowledge sources like Wikipedia, and limited adaptability for real-time or multimodal applications.

2. Iqbal H. Sarker (2024) [10] presents a position paper that critically evaluates the potential and risks of large language models from a trustworthy AI perspective. The methodology covers model selection, fine-tuning, evaluation, deployment, and post-deployment monitoring. The paper highlights key concerns such as misinformation, algorithmic bias, privacy violations, and ethical dilemmas. These issues threaten public trust and point to the urgent need for robust AI governance frameworks to ensure fairness, transparency, and accountability.

3. The comprehensive survey by Sumit Kumar Dam et al. (2024) [3] reviews the evolution and deployment of LLM-based AI chatbots, providing insights into their architecture and applications. Despite offering a wide-ranging view, the paper lacks in-depth discussion of critical issues such as ethical risks, misuse scenarios, and limitations in handling complex conversations. Moreover, it fails to suggest specific mitigation strategies or technical improvements, making it less actionable for developers or policymakers concerned with responsible AI use.

4. Yanwei Li et al. (2024) [6] propose Mini-Gemini, a Vision Language Model that introduces a specialized visual encoder and high-resolution visual token processing for improved multimodal understanding. The model achieves enhanced performance on tasks involving image-text reasoning, thanks to a finely curated dataset and advanced training strategies. However, its sophisticated structure and computational demand make it unsuitable for resource-limited environments, such as audio-only or low-power applications, reducing its scalability and broader usability.

5. In their study, Daye Nam et al. (2024) [8] explore how LLMs can assist in code comprehension through a tool called GILT. A user study comparing GILT with traditional web search among 32 participants revealed some improvements in user experience but no statistically significant difference in task completion time or comprehension accuracy. Additionally, results varied widely between novices and professionals, suggesting that the tool's effectiveness is context-dependent and may not offer universal benefits.

6. Oguzhan Topsakal and Tahir Cetin Akinci (2023) [11] present LangChain, a modular framework for building applications with LLMs. The platform facilitates rapid development through reusable components like prompts, chains, and memory modules, streamlining tasks such as summarization and question answering. However, LangChain inherits common LLM limitations, including generation inaccuracies and high latency due to repeated API interactions. These constraints can impair responsiveness and efficiency in real-world applications.

7. Yuqing Wang and Yun Zhao (2023) [12] assess the commonsense reasoning abilities of LLMs and multimodal LLMs using twelve diverse datasets. Their methodology involves both zero-shot and few-shot evaluation to analyze the integration of textual and visual cues. While the results reveal encouraging performance trends, the study is limited by its exclusive focus on English-language datasets, making its conclusions less generalizable to multilingual or culturally diverse contexts. Additionally, performance may vary as models evolve or are fine-tuned further.

8. The paper by Pooyan Rahmanzadehgervi et al. (2024) [9] critically evaluates the foundational capabilities of Vision Language Models using BlindTest, a benchmark consisting of simple geometric reasoning tasks. Despite their impressive results on complex vision-language tasks, the evaluated VLMs showed poor performance—averaging just 58.57 Percent accuracy—on low-level vision challenges like counting and shape recognition. These findings expose a significant blind spot in current VLM architectures and suggest the need for rethinking model training strategies.

9. Shreya Bhatia et al. (2024) [2] conduct a comparative performance analysis of generative AI tools for unit test generation across 109 Python projects. The study evaluates various tools based on code structure, complexity, and dependencies while excluding NumPy-based projects due to tool limitations. Although the analysis is methodical, it suffers from dataset constraints, potential selection bias, and a lack of diversity in sample size, particularly in more complex categories, which weakens the generalizability of its findings.

10. Toufique Ahmed and Premkumar Devanbu (2022) [1] explore the effectiveness of few-shot learning for code summarization using Codex. Their experiments, conducted on the CodeXGLUE benchmark, demonstrate that few-shot methods outperform zero- and one-shot approaches in generating accurate summaries. However, the research is limited by Codex's 4000-token input restriction, access limitations, and risks of overfitting, such as catastrophic forgetting. These issues constrain the scale and flexibility of experimentation and limit the method's application to large or complex projects.

Table 2.1: Comparative Analysis of Literature Survey

Sr. No	Title	Author(s)	Year	Methodology	Drawback
1	Retrieval Augmented Generation for Knowledge-Intensive NLP task	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, Douwe Kiela	2020	The methodology combines seq2seq models with Dense Passage Retrieval (DPR) to enhance language generation. It uses RAG-Token for autoregressive beam search and RAG-Sequence with thorough or fast decoding for efficient text generation. [5]	The RAG model relies heavily on external knowledge (Wikipedia), which limits scope and incurs high computational costs for retrieval. It reduces but doesn't eliminate hallucinations, lacks full interpretability, and struggles with real-time updates or multimodal inputs.
2	LLM potentiality and awareness: a position paper from the perspective of trustworthy and responsible AI modeling	Iqbal H. Sarker	2024	The methodology includes defining tasks and acquiring data, selecting and fine-tuning models, and evaluating their performance with metrics. Post-evaluation, models are deployed and monitored to ensure reliability. [10]	The paper highlights LLMs' susceptibility to generating misinformation, biases, and adversarial attacks, while also being opaque in decision-making and raising privacy and ethical concerns. These issues challenge trust, fairness, and responsible deployment in AI systems.
3	A Complete Survey on LLM-based AI Chatbots	Sumit Kumar Dam, Choong Seon Hong, Yu Qiao, Chaoning Zhang	2024	The paper surveys the evolution, applications, and challenges of LLM-based chatbots, from early models to current systems like ChatGPT. It aims to provide a comprehensive overview and explore future improvements for these technologies. [3]	This paper does not adequately address specific challenges faced by LLM-based chatbots, such as ethical concerns and data biases, nor does it discuss technical limitations like scalability and complex conversation handling. It lacks concrete examples or solutions related to the misuse of generated knowledge.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
4	Mini-Gemini: Mining the Potential of Multi-modality Vision Language Models	Yanwei Li1*, Yuechen Zhang1*, Chengyao Wang1*, Zhisheng Zhong1, Yixin Chen1, Ruihang Chu1, Shaoteng Liu1, Jiava Jia1,	2024	The methodology of Mini-Gemini focuses on improving Vision Language Models (VLMs) by enhancing visual tokens through an additional visual encoder for high-resolution refinement, constructing a high-quality dataset that supports precise image comprehension and reasoning-based generation, and leveraging VLM-guided generation techniques. [6]	Mini-Gemini's emphasis on high-resolution visual tokens, complex architecture, and computational demands makes it overly resource-intensive and ill-suited for audio-centric projects like yours, where simpler and more focused frameworks such as Whisper and Sentence-BERT are more effective.
5	Using an LLM to Help With Code Understanding	Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, Brad Myers	2024	A user study with 32 participants compared GILT to web search, analyzing task completion and code comprehension. Quantitative and qualitative data were collected. [8]	No significant improvement in task completion time or understanding. Benefits varied between students and professionals.
6	Creating Large Language Model Applications Utilizing LangChain: A Primer on Developing LLM Apps Fast	Oguzhan Topsakal, and Tahir Cetin Akinci	2023	LangChain, a framework designed to build AI applications utilizing large language models (LLMs). It uses modular components like prompts, chains, and memory to develop customizable pipelines that interact with external data sources for tasks like question-answering and document summarization. [11]	LangChain's reliance on LLMs means it may still face common issues like generating inaccurate outputs. Additionally, handling complex tasks can require multiple API calls, which may lead to slower execution or increased computational costs.

Sr. No	Title	Author(s)	Year	Methodology	Drawback
7	Gemini in Reasoning: Unveiling Commonsense in Multimodal Large Language Models	Yuqing Wang, Yun Zhao	2023	Four LLMs and two MLLMs are tested on 12 commonsense reasoning datasets using zero-shot and few-shot learning to assess accuracy, with focus on visual-textual integration. [12]	The study is limited to English datasets, missing cross-cultural nuances, and results may vary with future model updates or new datasets.
8	Vision language models are blind	Pooyan Rahmanzadehgervi, Logan Bolton, Mohammad Reza Taesiri, Anh Totti Nguyen	2024	The BlindTest benchmark evaluates four top Vision Language Models (VLMs) on seven low-level visual tasks, such as counting shapes and identifying intersections, focusing on simple geometric challenges. [9]	VLMs perform poorly on basic visual tasks, with an average accuracy of 58.57.
9	Unit Test Generation using Generative AI: A Comparative Performance Analysis of Autogeneration Tools	Shreya Bhatia, Tarushi Gandhi, Dhruv Kumar Pankaj Jalote	2024	Compared unit test generation tools using Python code samples categorized by structure. Evaluated tools on 109 Python projects, excluding NumPy-dependent ones. Selected core modules based on LOC, complexity, and import dependencies. [2]	Limited by Pynguin's inability to handle NumPy projects, few large samples in Category 1, and potential bias in file selection.
10	Few-shot training LLMs for project-specific code-summarization	Toufique Ahmed, Premkumar Devanbu	2022	The methodology - used Codex for few-shot code summarization. Structured prompts with code-summary pairs and a target function. Evaluated on CodeXGLUE using 1,000 samples due to limited Codex access. [1]	Limited access to the Codex model constrained the dataset size and overall experimentation. The 4000-token limit for few-shot training restricted the number of sequences, and using too much data could lead to catastrophic forgetting. Zero-shot and one-shot training provided lower performance compared to few-shot methods.

Chapter 3

Project Design

Project design is the foundational phase in which the structure and execution strategy of a project are planned in detail. It involves identifying the problem the project aims to solve, defining clear objectives, and determining the scope and limitations of the system. This stage lays out the methodology to be used, the system architecture, the flow of data, and the interaction between different components through visual representations like UML diagrams, data flow diagrams, and sequence diagrams. Additionally, it includes the selection of suitable tools, technologies, and platforms for development, along with a well-defined timeline for execution.

3.1 Existing System

In the realm of artificial intelligence, various systems have been developed to handle diverse applications such as Retrieval-Augmented Generation (RAG), chatbot-based interactions, and vision-based AI models. Existing systems primarily follow modular architectures that integrate user input processing, knowledge retrieval, and intelligent response generation. Chatbot frameworks like Dialogflow and Rasa implement natural language processing (NLP) for conversational AI, while multimodal AI systems incorporate image, audio, and text-based inputs for enhanced understanding.

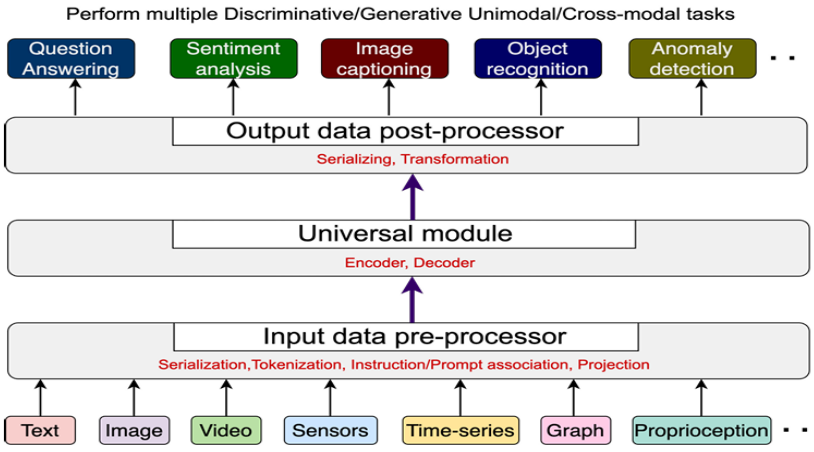

Figure 3.1: A generalist multimodal architecture pipeline

The Figure 3.1 from [7] is system architecture represents a multi-modal AI framework capable of handling diverse discriminative and generative tasks across various data types. It consists of three main layers: the Input Data Pre-Processor, which processes different input modalities such as text, images, videos, sensors, time-series data, graphs, and proprioception using functions like serialization, tokenization, prompt association, and projection to standardize the data format. The Universal Module acts as the core processing unit, utilizing encoder-decoder mechanisms to transform and interpret input data efficiently for further processing. Finally, the Output Data Post-Processor applies serialization and transformation techniques to generate meaningful outputs tailored to specific AI tasks, including question answering, sentiment analysis, image captioning, object recognition, and anomaly detection. This system enables seamless integration of multiple data types, facilitating advanced AI-driven decision-making.

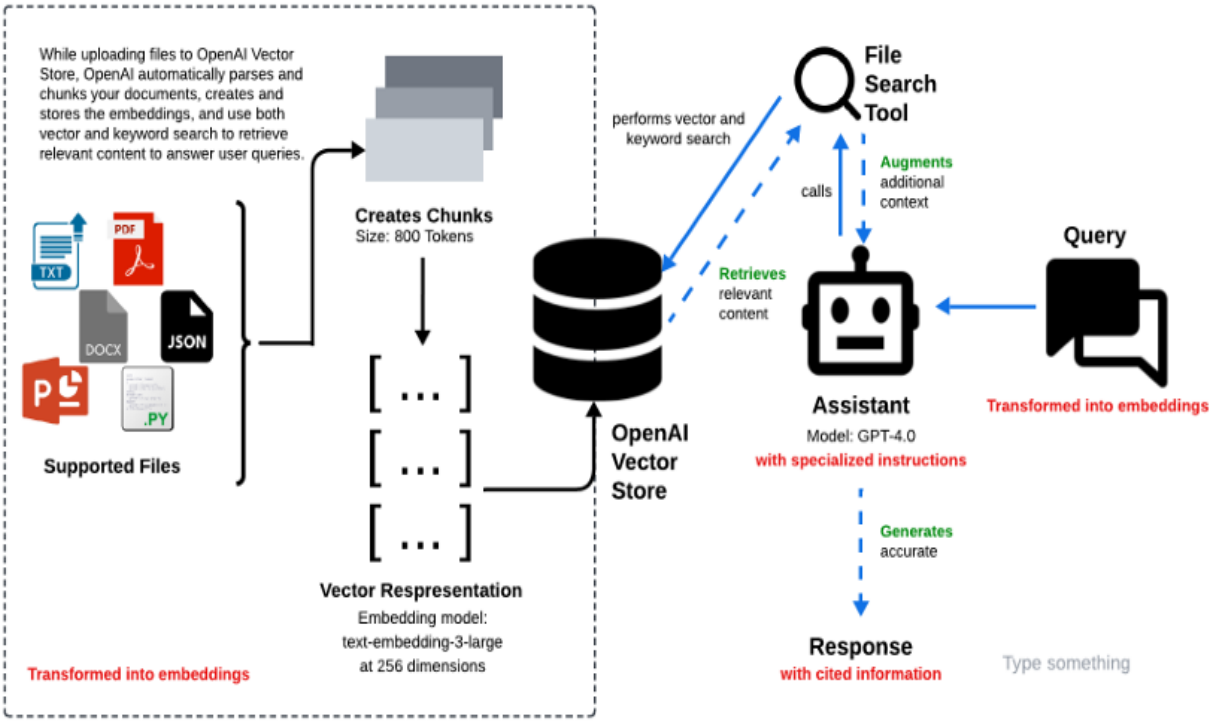

Figure 3.2: A generalist multimodal architecture pipeline

The Figure 3.2 from [4] illustrates a document-based AI assistant powered by OpenAI's vector store and GPT-4.0 for intelligent search and response generation. It begins with document ingestion, where supported file types such as TXT, PDF, DOCX, JSON, and Python scripts are uploaded. The system automatically parses and chunks the documents into segments of 800 tokens before converting them into vector embeddings using the text-embedding-3-large model at 256 dimensions. These embeddings are stored in the OpenAI Vector Store, allowing both vector and keyword-based searches to efficiently retrieve relevant content. When a user submits a query, it is also transformed into embeddings and processed through a file search tool, which augments additional context by retrieving the most relevant content from the stored embeddings. This information is then passed to the AI assistant (GPT-4.0 with specialized instructions), which generates accurate responses with cited information.

3.2 Proposed System Architecture

The system architecture of LUMIS (LLM-Based Unified Multimodal Intelligent System) 3.3 is designed to support seamless multimodal interaction by intelligently processing diverse input formats using state-of-the-art language models. At its core, the architecture follows a modular pipeline that begins with input classification, routes data through appropriate Gemini models based on content type, and delivers context-aware, accurate responses to users via a dynamic user interface. This architecture ensures scalability, flexibility, and high performance, making LUMIS capable of handling complex real-world tasks ranging from code analysis and document summarization to audio transcription and image interpretation.

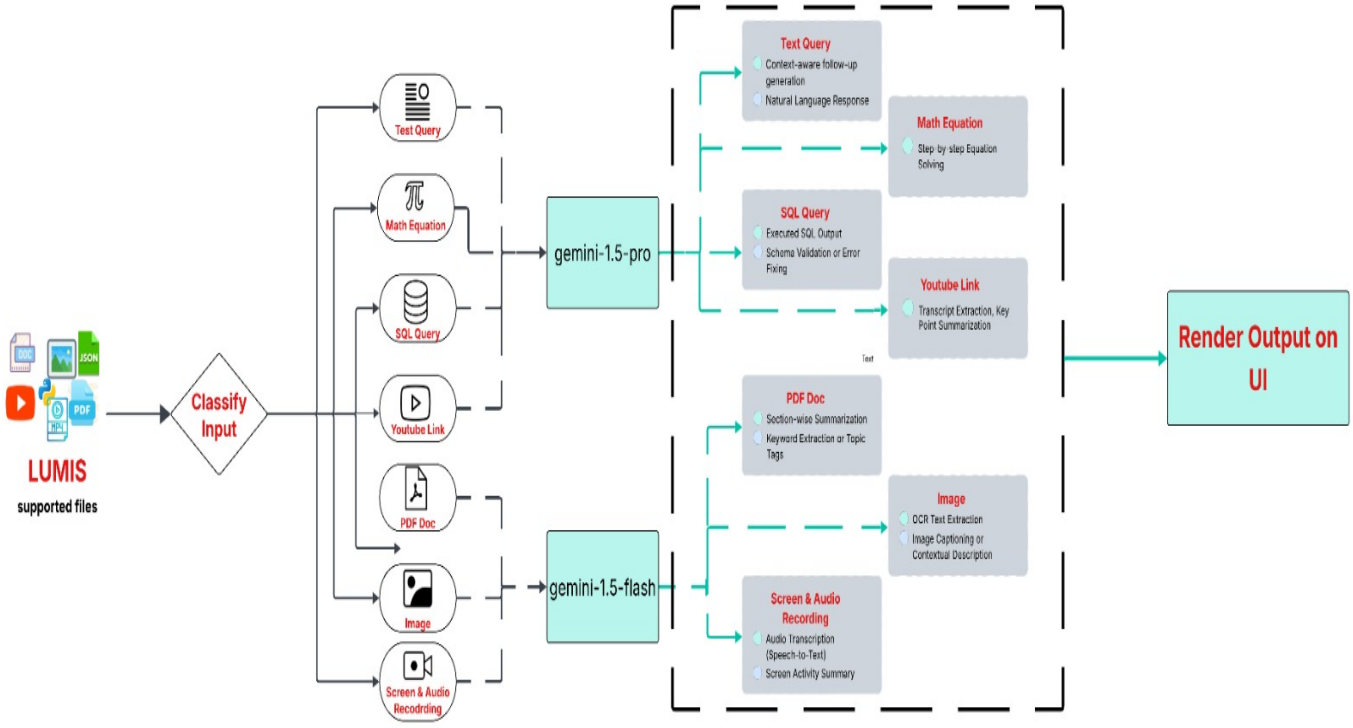

Figure 3.3: Proposed System Architecture of Lumis

LUMIS (LLM-Based Unified Multimodal Intelligent System) is a robust and versatile AI-powered framework designed to handle and intelligently process various forms of input data. The system supports a wide range of file types, including documents (DOC, PDF), images, JSON, HTML, YouTube links, and even screen/audio recordings. At its core, LUMIS is built to automatically classify the type of input it receives, delegate processing to the appropriate Gemini model (either gemini-1.5-pro or gemini-1.5-flash), and render the final output onto the user interface. This intelligent classification and modular processing architecture makes LUMIS a truly multimodal assistant capable of adapting to different user needs.

The journey begins when a user submits a file or query. The system features an input classification mechanism that identifies the nature of the input—whether it is a text query, mathematical equation, SQL query, YouTube link, PDF document, image, or screen/audio recording. This classification is crucial as it determines the appropriate model and processing

flow for the input. For instance, text-heavy and context-rich tasks like question answering or SQL query debugging are better handled by gemini-1.5-pro, while image-related or fast-response tasks such as OCR and audio transcription are routed to the more lightweight gemini-1.5-flash model.

Inputs such as text queries, math equations, SQL queries, YouTube links, and PDFs are handled by the gemini-1.5-pro model, which is known for its advanced reasoning and context-awareness. For text queries, it generates coherent and context-sensitive natural language responses, often capable of following up on previous queries. When it comes to mathematical equations, the model provides step-by-step breakdowns, making it ideal for educational or problem-solving purposes. For SQL queries, the system not only generates executed outputs but also performs schema validation and identifies or corrects logical or syntactical errors in the query. YouTube links are processed to extract transcripts and summarize key points, making lengthy videos quickly digestible. Lastly, PDF documents are subjected to section-wise summarization and keyword or topic tag extraction to make dense documents easier to navigate and understand.

The gemini-1.5-flash model is optimized for faster and lightweight tasks, which is ideal for real-time or visually-oriented inputs like images and screen/audio recordings. When an image is inputted, the model performs OCR (Optical Character Recognition) to extract embedded text and can also generate contextual image descriptions or captions. This is particularly useful in scenarios like document digitization or visual content analysis. For screen and audio recordings, the model transcribes spoken audio into text using speech-to-text technology and summarizes on-screen activities, thus offering a comprehensive overview of what occurred during a session. This feature is useful in automated meeting summaries, user session tracking, or even debugging.

After processing the input using the appropriate model, the final step involves rendering the output on the user interface. Regardless of the input type, the system ensures that the generated result—whether it is a natural language response, SQL execution output, image caption, document summary, or transcription—is clearly and effectively presented to the end user. The UI layer is designed to be dynamic and adjusts its presentation format based on the output type, ensuring a user-friendly and coherent experience across various tasks and modalities.

In essence, LUMIS serves as a comprehensive AI assistant that leverages the strength of Google's Gemini models to intelligently interpret, analyze, and respond to a wide range of input formats. Its architecture is built for flexibility, scalability, and usability, making it suitable for academic, professional, and personal applications. By incorporating both gemini-1.5-pro and gemini-1.5-flash, LUMIS optimally balances depth of reasoning with processing speed, effectively covering all aspects of multimodal interaction and intelligent automation.

3.3 UML Diagrams

Unified Modeling Language (UML) is a standardized general-purpose modeling language used to specify, visualize, construct, and document the artifacts of a software system. UML (Unified Modeling Language) diagrams were used to visually represent the structure, behavior, and interactions within the system, ensuring clarity in both design and implementation. The Use Case Diagram outlines the system's primary functions and the interactions between users (such as end-users and administrators) and features like "Upload File," "Generate SQL Query," and "Visualize Data," offering a high-level functional overview. The Activity Diagram captures the dynamic workflow of the system, detailing the step-by-step process from user login or file upload to the generation and visualization of results, including decision points and parallel activities. The Sequence Diagram illustrates the chronological flow of messages and interactions between system components such as the user interface, processing engine, and database, highlighting the communication pattern for operations like query execution and result retrieval. Collectively, these diagrams provide a comprehensive understanding of the system's operation, improving both development and future scalability.

3.3.1 Activity Diagram

The Activity Diagram for L.U.M.I.S, as illustrated in Figure 3.4, presents a structured overview of how the system processes various forms of user input in a sequential and intelligent manner. It begins with the user providing an input, which can range from simple text queries, SQL statements, and YouTube links to more complex multimodal data such as screen recordings with audio, PDF documents, hand-drawn equations via Air Canvas, or static images. The system first identifies the nature of the input through a classification step, determining its type and content. Based on this analysis, L.U.M.I.S routes the input to the appropriate AI-powered module—such as a summarization engine, code analysis tool, RAG (Retrieval-Augmented Generation) pipeline, or a visual processing component. For example, a SQL input triggers automated query generation and execution; a YouTube link engages transcription and summarization; and handwritten equations are converted and interpreted through symbolic math engines.

This intelligent routing reflects L.U.M.I.S's modular and flexible design, where each input is matched with a specialized capability. OCR is used to extract text from images or PDFs, NLP modules handle question-answering and summarization, and speech-to-text models process spoken content. The system seamlessly integrates these diverse functionalities, ensuring that each task is handled in a context-aware and coherent manner. The activity diagram captures this end-to-end flow—from initial input recognition through processing and task execution to final output generation. It not only illustrates the adaptability of L.U.M.I.S across various input types but also emphasizes the unified, multimodal nature of the platform, showcasing its ability to operate as an intelligent assistant across textual, visual, auditory, and interactive domains.

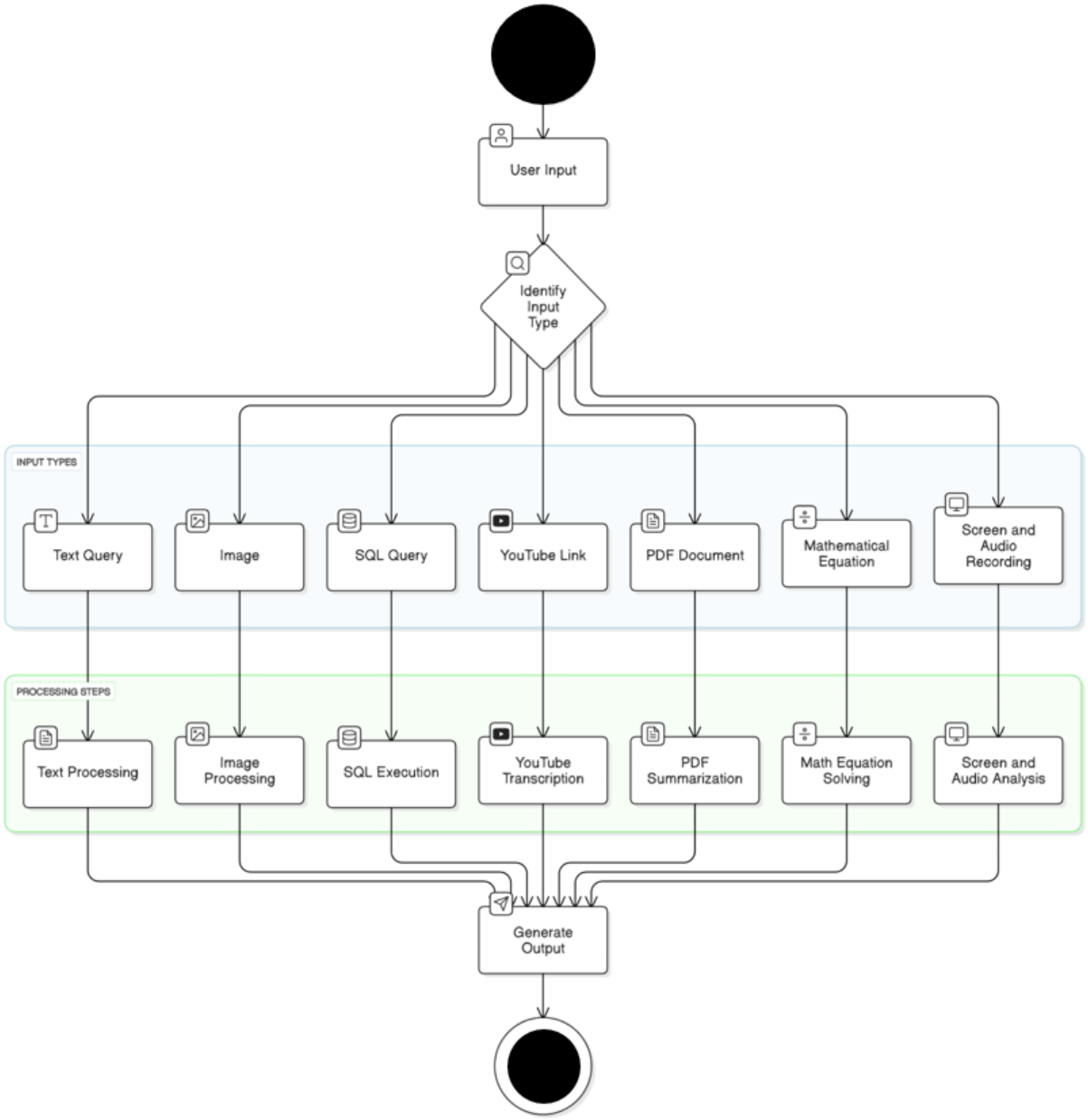

Figure 3.4: Activity Diagram of Lumis

The Activity Diagram of L.U.M.I.S outlines the systematic flow of operations starting from diverse user inputs to final output generation. The process initiates with users providing input in various forms such as image uploads, screen recordings with audio, text queries, YouTube links, PDF files, mathematical equations, or SQL queries. A decision node evaluates the type of input and routes it to the corresponding processing module—for instance, screen and audio analysis, text processing, image analysis, SQL execution, YouTube transcription and summarization, PDF summarization, or mathematical equation solving. Each module utilizes dedicated AI-driven techniques to analyze the input and generate contextually relevant output. The system then channels these outputs—such as debugging insights, visualizations, conversational responses, or equation and SQL results—into a dynamic user interface. This structured, multimodal activity flow ensures intelligent, efficient, and user-friendly interaction, reflecting L.U.M.I.S's core design principles of adaptability.

3.3.2 Use Case Diagram

The Figure 3.5 use case diagram illustrates the key interactions between the primary actors—Users and the System Admin—and the core functionalities of the LUMIS system. The diagram maps out how users interface with a range of AI-powered services such as Transcription, Image-to-Text Conversion, Real-time Code Assistance, and Text/SQL Generation. Advanced modules like the Tapas Model offer intelligent support for data interpretation, while the Air Canvas enables dynamic content creation through gesture-based mathematical input. These use cases collectively reflect LUMIS's commitment to delivering an intuitive and intelligent multimodal user experience

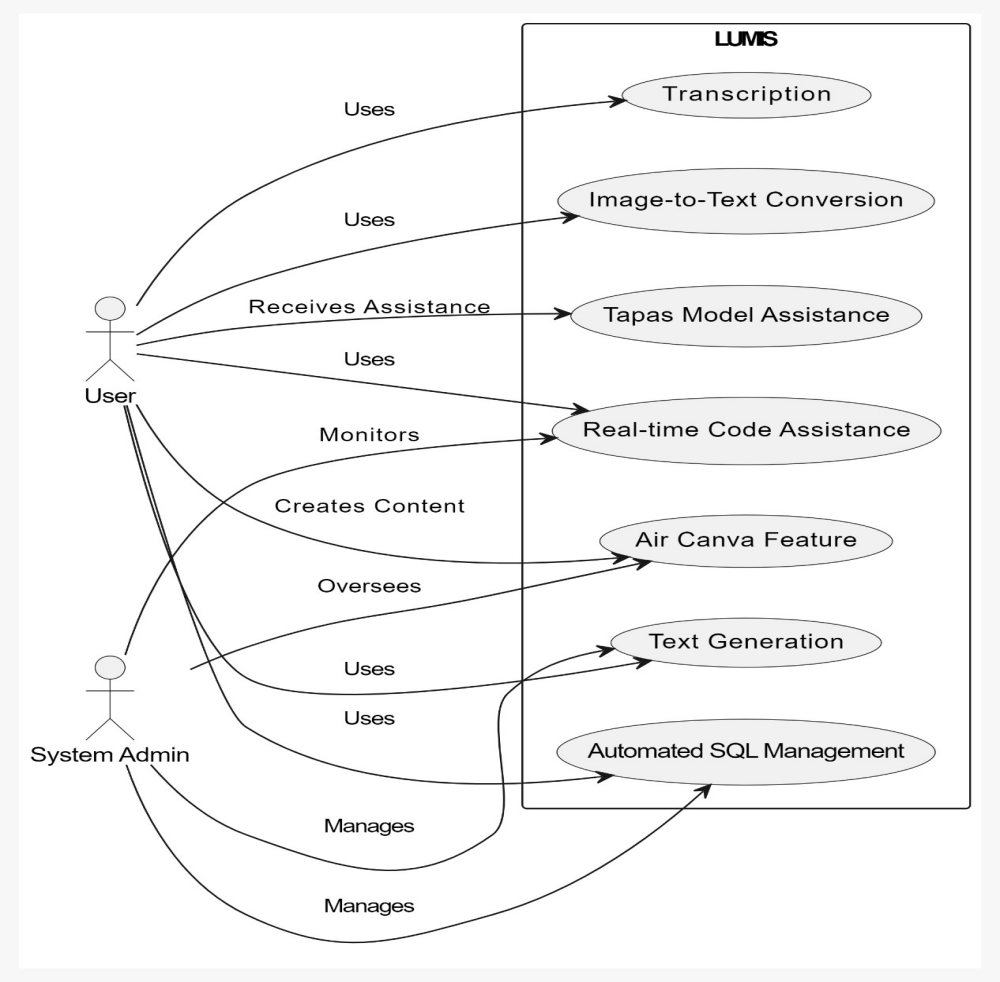

Figure 3.5: Use Case Diagram of Lumis

Overall, the use case diagram encapsulates the comprehensive utility of LUMIS by showcasing how users can seamlessly interact with its features to accomplish diverse tasks, ranging from code debugging to document summarization. The System Admin ensures backend management and system integrity. This high-level overview of actor-system interactions emphasizes the platform's flexibility, modularity, and its capability to enhance productivity through AI-driven automation and intelligent task handling.

3.3.3 Sequence Diagram

A sequence diagram is a visual representation of how different components of a system interact with each other over time. It shows the order in which messages or events are exchanged between various objects or entities, illustrating how data flows through the system. The diagram typically includes actors (external entities interacting with the system), objects (system components involved in the interaction), and the messages they exchange. Each interaction is represented by arrows, with the direction indicating the flow of communication. Sequence diagrams are commonly used to model the dynamic behavior of a system, providing insight into how different parts of the system collaborate to perform a particular function or process. This makes them valuable tools in understanding and designing systems, particularly for visualizing time-based interactions and identifying the sequence of operations.

Image and Text Processing Module

The Image and Text Processing Module 3.6 in LUMIS serves as a powerful multimodal interface where users can analyze visual and textual content together or separately. Upon launching this feature through the Streamlit UI, users are prompted to either upload an image file, input some text, or provide both. The system collects this input and forwards it to the Gemini API, which is designed to handle diverse data types simultaneously. If only text is provided, the API performs tasks such as summarization, language analysis If an image alone is submitted, the API might identify objects, interpret scenes, or perform OCR (Optical Character Recognition). When both an image and text are provided, the Gemini model uses its multimodal reasoning capabilities to interpret the relationship between the two for example, generating a descriptive caption for the image based on the text correctly describes the visual content. The output is then returned to the front end and rendered for the user in a clear manner offering insightful interpretations in real time.

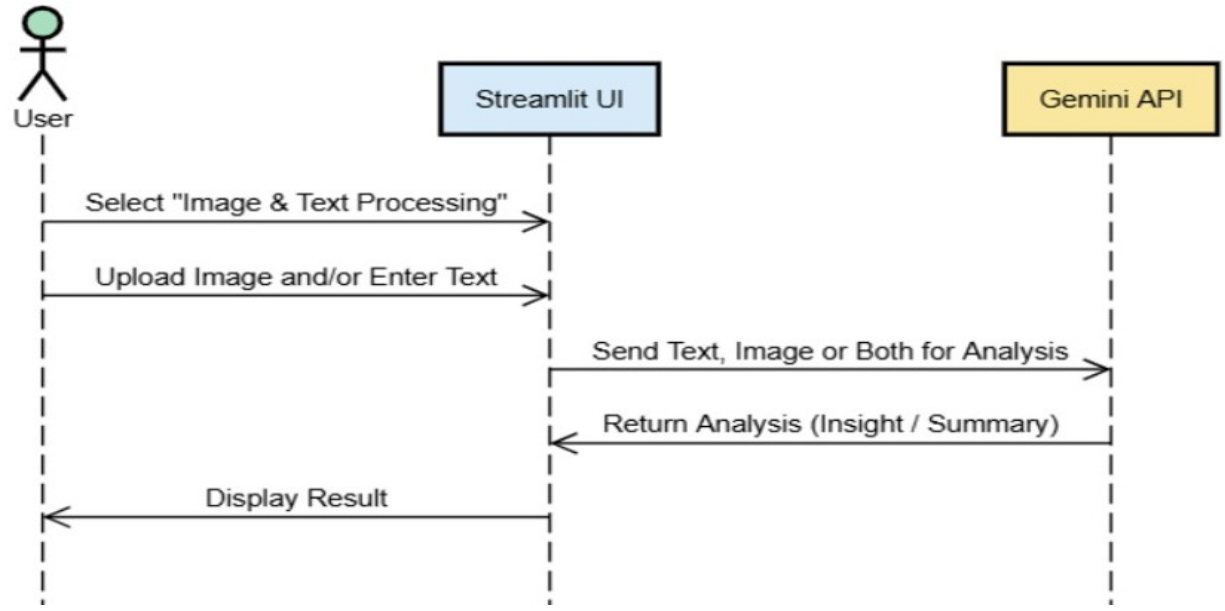

Figure 3.6: Sequence Diagram of Image and Text Processing Module

T.A.P.A.S Module

The TAPAS Module 3.7, which stands for Technical Assistance Platform for Advanced Solution, is one of the most interactive features in LUMIS. It enables users to record technical sessions, walkthroughs, or tutorials in real-time, analyze their content, and receive improvement suggestions. When the user activates TAPAS via the UI and hits "Start Recording", a backend controller begins capturing audio and video input from the user's system. The video and audio streams are saved locally in .mp4 and .wav formats, respectively. Once the session is complete, these media files are merged into a single playback file using tools like FFmpeg. The audio component is isolated and sent to the Gemini API for transcription. Once the transcript is returned, both it and the video are analyzed to identify technical explanations, potential bugs, coding issues, and communication clarity. The results are then displayed back to the user, including the full transcript, flagged sections, and improvement tips, making it an invaluable tool for educators, developers, and content creators.

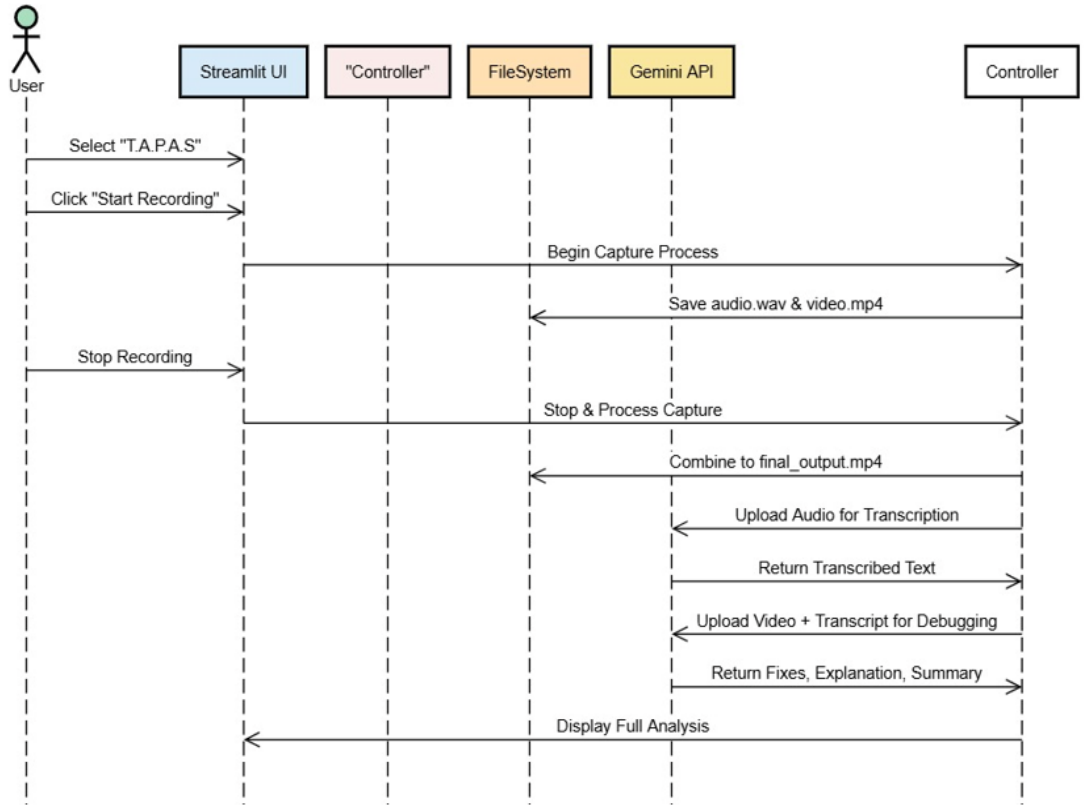

Figure 3.7: Sequence Diagram of T.A.P.A.S Module

S.Q.L.A.G.E Module

The SQLAGE Module 3.8, short for "SQL Automatic Generation and Execution", allows users to interact with databases using natural language. The process begins when a user selects this feature on the Streamlit UI and enters a query in plain English, such as "Show me the total number of users from each country." The system then uses SQLAlchemy to connect to a MySQL database and reflect its schema — meaning it retrieves all table names, fields, and data types, essentially building a blueprint of the database structure. This schema, along with the user's query, is then passed to the Gemini API, which interprets the intent behind the natural language and formulates a valid SQL query. Once the SQL is generated, it is executed against the live database, and the results — typically tabular data — are fetched and displayed to the user within the UI. This module significantly lowers the technical barrier for users who need database insights but may not be proficient in SQL.

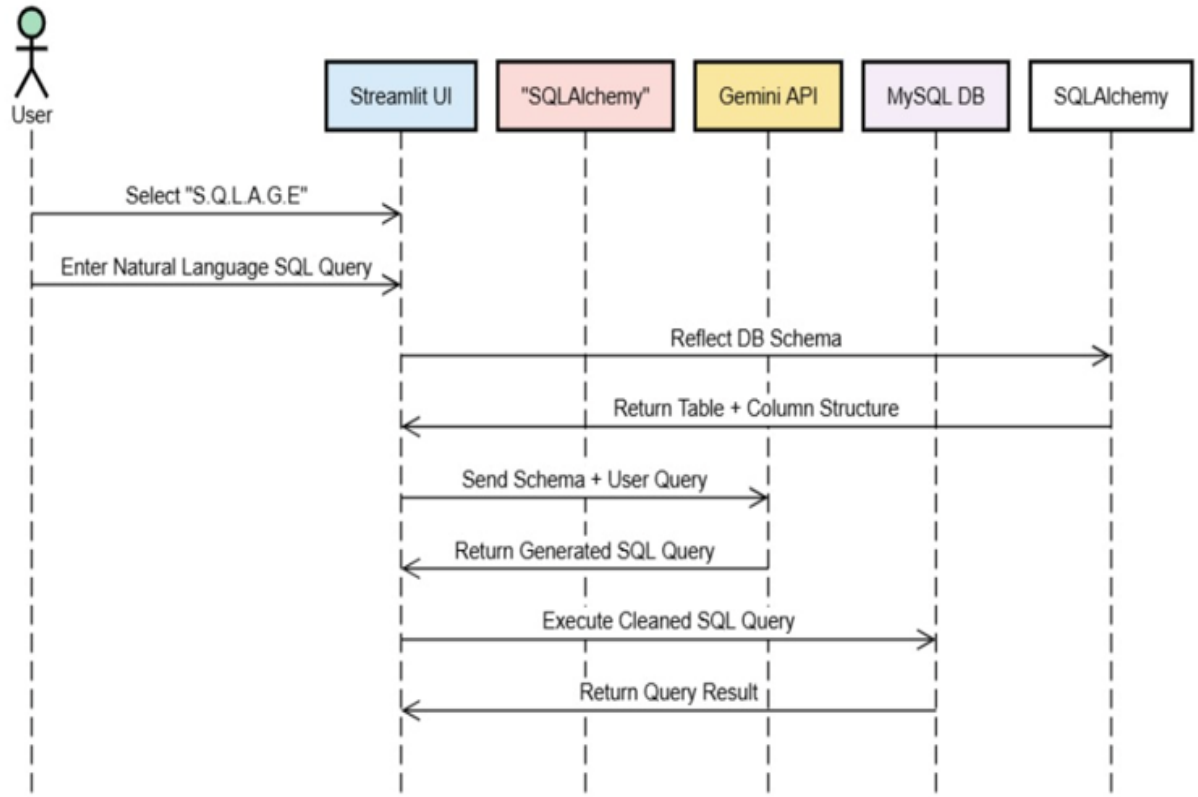

Figure 3.8: Sequence Diagram of S.Q.L.A.G.E Module

PDF RAG Module

The PDF Summarization Module 3.9 is designed to help users efficiently extract meaningful information from large volumes of documents. Upon selecting this feature, users can upload multiple PDFs via the Streamlit interface and optionally include a question related to the document content. These PDFs are temporarily stored in the local file system, allowing the system to manage and read them easily. Once stored, the backend processes each PDF to extract text, which is bundled along with the user's query and sent to the Gemini API for summarization or question answering. The Gemini model then parses through the textual data, breaks it into manageable chunks if necessary, and constructs a concise, context-aware response. This summary or direct answer is finally sent back to the UI and displayed in an easily readable format. This module is especially helpful for users dealing with large academic, legal, or technical documents, where quick comprehension is essential.

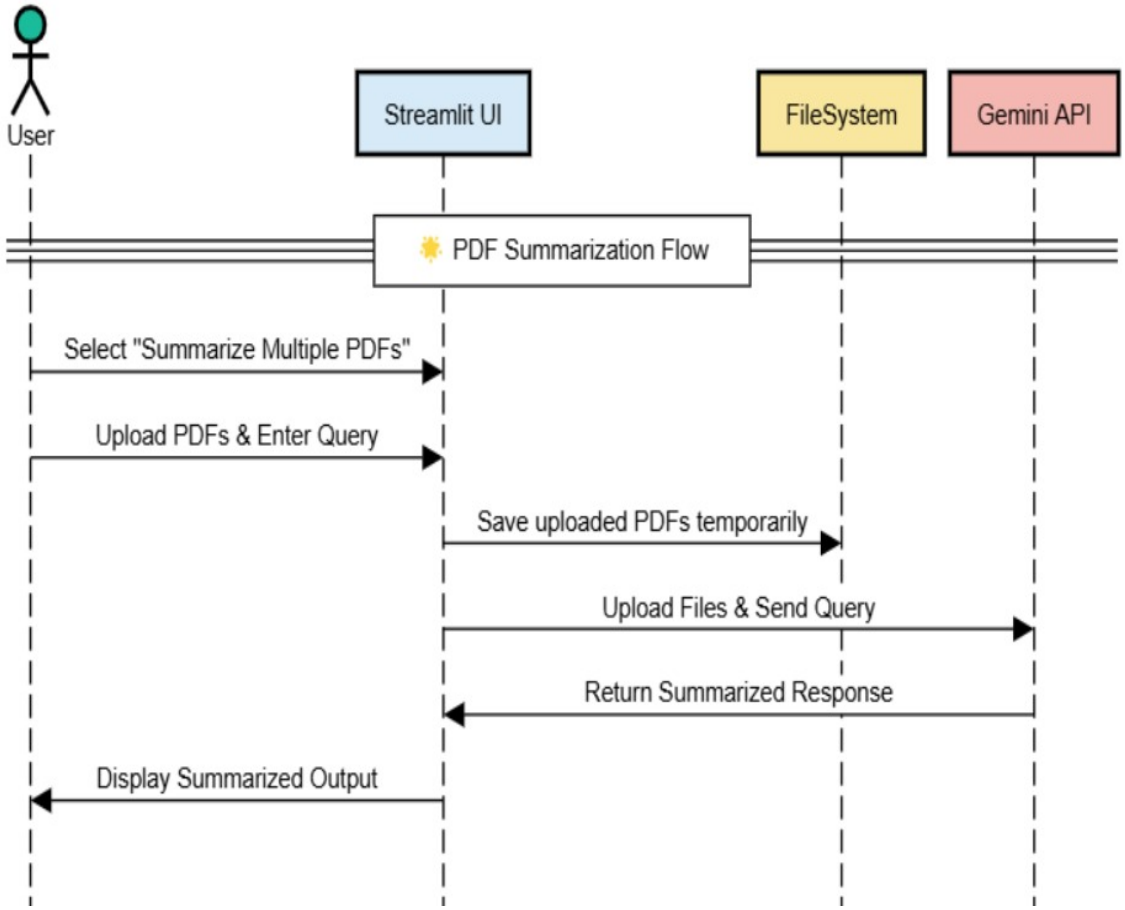

Figure 3.9: Sequence Diagram of PDF RAG Module

YT Transcription Module

The YouTube Summarizer Module 3.10 simplifies the process of consuming long-form video content. When a user inputs a YouTube video URL into the Streamlit interface, the system extracts the video ID and uses the YouTube Transcript API to retrieve the video's subtitles or spoken text. If the transcript is available, the system then breaks the full text into smaller segments to fit the processing limits of the Gemini model. Each segment is processed individually by Gemini, which produces partial summaries that are eventually combined into a complete overview. The final result is a well-organized, easy-to-read summary of the video's key points, which is then displayed back to the user through the UI. This module is especially useful for students, researchers, and professionals who need to understand video content quickly without watching the entire thing.

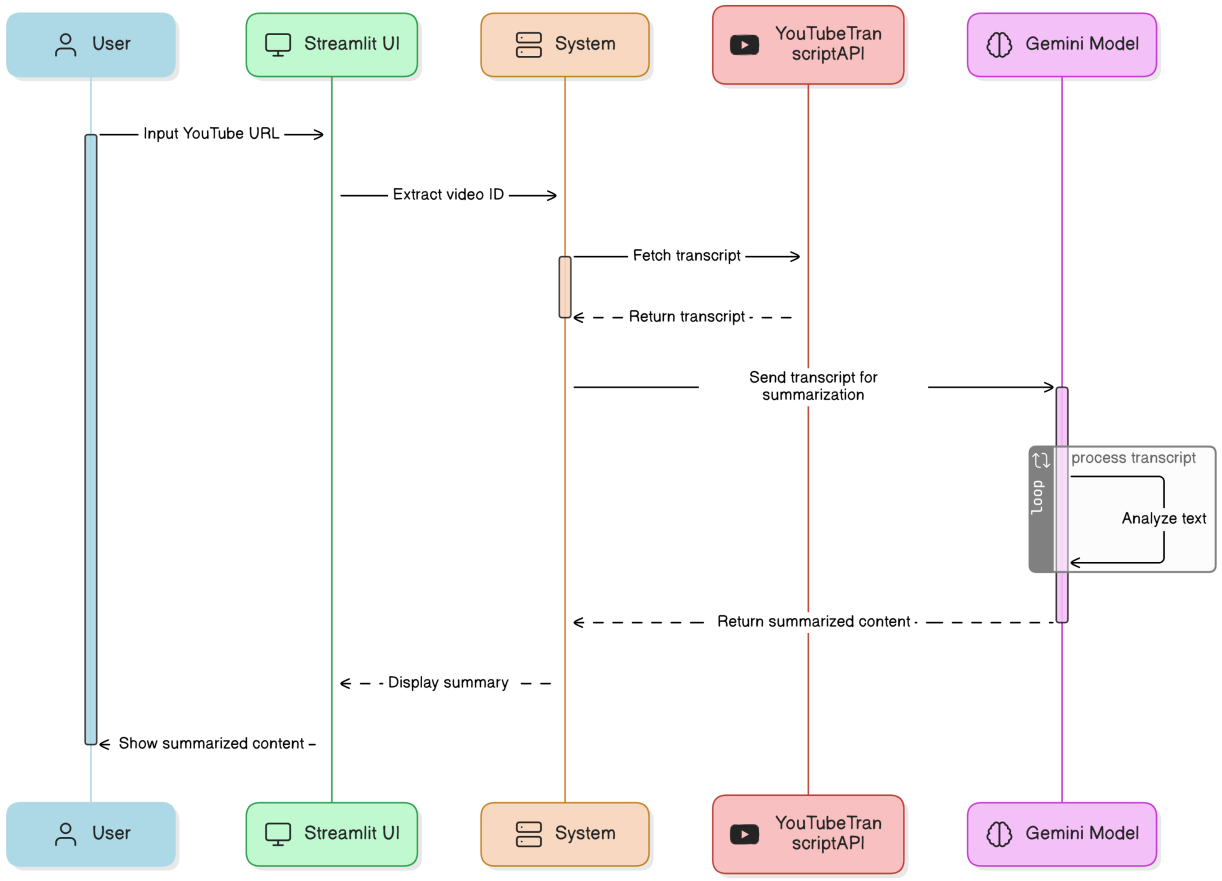

Figure 3.10: Sequence Diagram of YT Transcription Module

Chapter 4

Project Implementation

The LUMIS system is designed to integrate advanced AI capabilities with user-friendly interfaces to streamline various tasks involving content analysis, summarization, and information retrieval. The project leverages Google's Gemini AI models to process a range of content types, including PDF documents, images, text, and YouTube video transcripts. By automating tasks like summarization and extraction of key insights, LUMIS significantly enhances productivity across various fields such as research, education, and content analysis. The following code snippets demonstrate the core functions of the system, showcasing its ability to handle different input types, execute intelligent queries, and provide meaningful, context-aware responses

4.1 Code Snippets

The following functions leverage Google's Gemini AI models to process and analyze a wide variety of content, including PDF documents, images, text, and YouTube video transcripts. These functions are designed to automate the tasks of summarization and extraction of key insights, making information retrieval faster, more efficient, and more accessible. For instance, when dealing with PDF documents, the system is capable of extracting relevant information from long, complex files, distilling them into concise summaries that capture the essence of the content. Similarly, in the case of images, the AI can perform optical character recognition (OCR) to read and analyze text within images, transforming them into actionable data. This is particularly useful for processing scanned documents or images that contain textual information, such as charts, diagrams, or handwritten notes.

Moreover, the system integrates advanced natural language processing (NLP) techniques to analyze text input, automatically summarizing large volumes of information and extracting crucial points, insights, or concepts from the provided text. This makes it particularly valuable for content-heavy tasks, like processing research papers, reports, or technical documentation, where quick understanding and summarization are required. Additionally, the system can process YouTube video transcripts by extracting the text from the video's captions or transcript, and then summarizing the core message. This enables fast access to key insights from long-form video content, which is especially useful for applications in education, content review, or digital marketing, where understanding video content quickly is essential.

```python
def summarize_pdfs(uploaded_files):
    model = genai.GenerativeModel("gemini-1.5-flash")
    summaries = {}
    for uploaded_file in uploaded_files:
        with pdfplumber.open(uploaded_file) as pdf:
            pdf_text = " ".join(page.extract_text() for page in pdf.pages if page.extract_text())
        response = model.generate_content(f"Summarize this document:\n{pdf_text}")
        summaries[uploaded_file.name] = response.text
    return summaries
```

Figure 4.1: PDF Summarization Function

The Figure 4.1 `summary_pdfs(uploaded_files)` function processes and summarizes the content of multiple PDF documents using Google's Gemini-1.5-Flash AI model. This function begins by initializing the AI model, which is specifically designed for high-performance document processing. It then iterates through each of the provided PDF files, utilizing the pdfplumber library to extract text from each page. The extracted text from all pages is concatenated into a single string, forming a continuous flow of content that is passed to the AI model. The model analyzes this concatenated text and generates a concise, coherent summary of the entire document, which is then returned as the output. The summaries are stored in a dictionary, mapping each file name to its corresponding summary, making it easy for users to track and retrieve the results. This function is particularly valuable for automating document processing in areas such as research, legal analysis, content review, and information retrieval, where large volumes of documents need to be analyzed quickly. By leveraging the power of AI, the system can efficiently extract key insights, eliminating the need for manual reading and analysis of lengthy texts. Furthermore, this functionality ensures faster decision-making, enabling professionals to access critical information much more efficiently.

```python
def process_image_text(text_input=None, image_file=None):
    model = genai.GenerativeModel('gemini-1.5-flash')
    if text_input and image_file:
        image = Image.open(image_file)
        response = model.generate_content([f"Analyze this image and text:\n{text_input}", image])
    elif image_file:
        image = Image.open(image_file)
        response = model.generate_content(["Analyze this image:", image])
    elif text_input:
        response = model.generate_content(f"Analyze this text:\n{text_input}")
    else:
        return "Please provide either text or an image."
    return response.text
```

Figure 4.2: Image Processing & Text Analysis Function

The Figure 4.2 `process_image_text(text_input=None, image_file=None)` function is a versatile tool that leverages Google's Gemini-1.5-Flash AI model to analyze and extract insights from text, images, or both. The function first checks whether both text and an image are provided; if so, it processes them together to gain insights from both sources simultaneously. If only an image is provided, it uses computer vision techniques and optical character recognition (OCR) to extract textual information from the image. If only text is provided, it analyzes the content using advanced natural language processing (NLP) to identify key points, summarize the content, or extract relevant data. If neither input is provided, the system prompts the user to enter either text or an image. This function is particularly valuable for applications in content analysis, document processing, and AI-driven multimedia analysis, as it allows users to efficiently process and derive meaningful insights from various formats of data. By combining text and image analysis, the function enhances automation in data interpretation and content evaluation, saving time and improving productivity for professionals working with diverse datasets, such as scanned documents, images containing charts or handwritten notes, and mixed media content.

```python
def process_youtube_video(youtube_url):
    try:
        video_id = youtube_url.split("v=")[-1].split("&")[0]
        transcript_data = YouTubeTranscriptApi.get_transcript(video_id)
        transcript_text = " ".join([item['text'] for item in transcript_data])

        model = genai.GenerativeModel('gemini-pro')
        response = model.generate_content(f"Summarize this transcript:\n{transcript_text}")
        return response.text
    except (TranscriptsDisabled, NoTranscriptFound):
        return "❌ No transcript available for this video."
    except Exception as e:
        return f"❌ Error: {str(e)}"
```

Figure 4.3: YouTube Video Transcript Summarization Function

The Figure 4.3 `process_youtube_video(youtube_url)` function extracts and summarizes a YouTube video transcript using Google's gemini-pro AI model. It retrieves the video ID from the URL, fetches the transcript via YouTubeTranscriptApi, and processes the text into a single string, which is then summarized by the AI model.This functionality is particularly useful for automating video content summarization in domains like education, research, content analysis, and digital marketing. By leveraging AI-driven summarization, it enables quick extraction of key insights, saving time and enhancing accessibility for developers and professionals working with large volumes of video content.

4.2 Steps to access the System

LUMIS provides a seamless and user-friendly interface for interacting with AI-powered tools. Users can select a task from the dropdown menu, such as multi-PDF summarization, chatbot interaction, or YouTube transcription. After uploading files or entering queries, the system processes the input and delivers accurate responses in real time. With its intuitive design and dynamic functionality, LUMIS enhances efficiency and simplifies complex tasks.

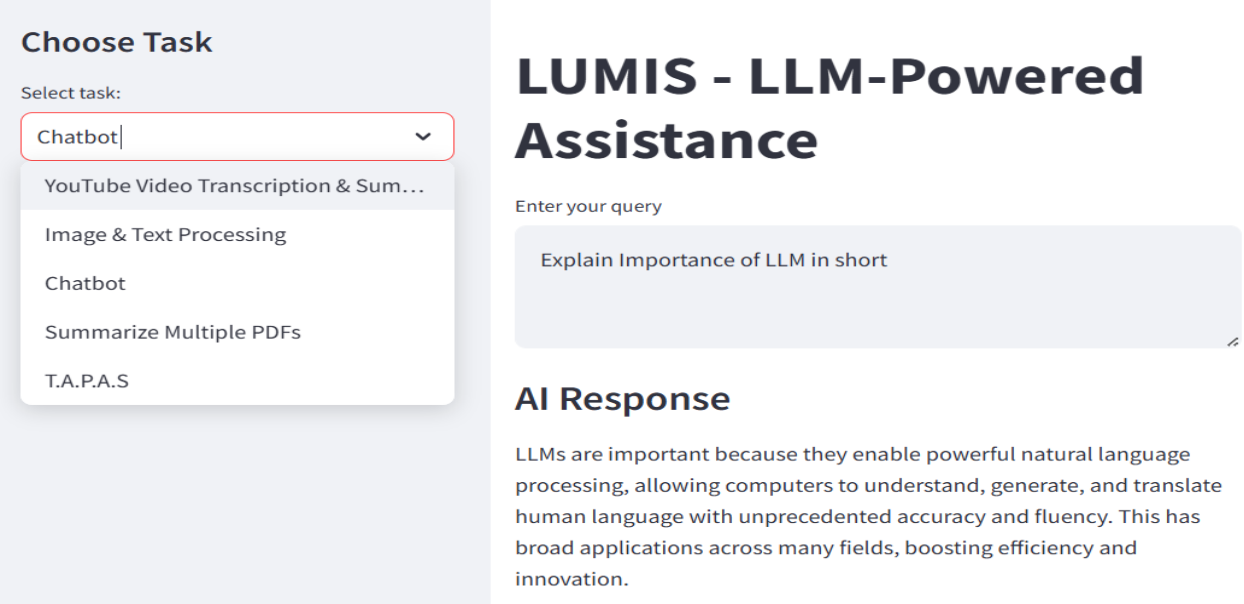

Figure 4.4: Summarize Multiple PDFs

The Figure 4.4 showcases the interface of "LUMIS – LLM-Powered Assistance," a tool designed for various AI-driven tasks. On the left side, a dropdown menu labeled "Choose Task" is visible, where the user has selected the "Chatbot" option. Other available tasks include YouTube video transcription and summarization, image and text processing, PDF summarization, and T.A.P.A.S. The right section features an input box where the user has entered the query, "Explain the importance of LLM in short." Below, the AI-generated response highlights the significance of Large Language Models (LLMs), stating that they enhance natural language processing by enabling computers to understand, generate, and translate human language with high accuracy and fluency. This has wide-ranging applications across industries, improving efficiency and fostering innovation.
Additionally, the chatbot module demonstrates LUMIS's capability to provide instant, coherent, and context-aware responses tailored to diverse queries. The fluid design of the interface supports real-time interaction, minimizing cognitive load for the user. This intuitive layout not only accommodates both technical and non-technical users but also ensures accessibility and usability. By integrating LLMs seamlessly into user workflows, LUMIS serves as a powerful productivity tool for education, customer service, research, and business intelligence applications.

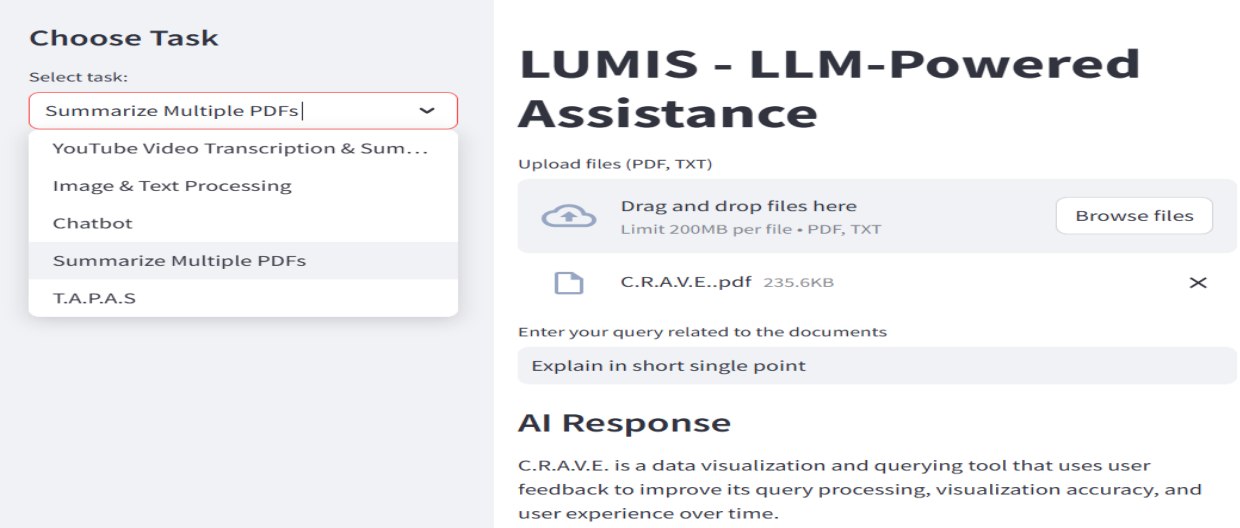

Figure 4.5: Chatbot

The Figure 4.5 displays an interface of "LUMIS – LLM-Powered Assistance," a tool designed for processing various tasks such as summarizing multiple PDFs, transcribing YouTube videos, image and text processing, and chatbot functionalities. The left section of the interface features a dropdown menu labeled "Choose Task," where the user has selected the "Summarize Multiple PDFs" option. The right section includes an upload panel allowing users to drag and drop files (PDF, TXT) with a size limit of 200MB per file. A file named "C.R.A.V.E..pdf" has been uploaded, and the user has entered a query requesting a brief summary. Below, an AI-generated response provides a concise explanation of the document, stating that C.R.A.V.E. is a data visualization and querying tool that enhances query processing, visualization accuracy, and user experience through feedback. The interface is structured to facilitate efficient document processing and summarization, enhancing accessibility and usability for users handling large text-based files.

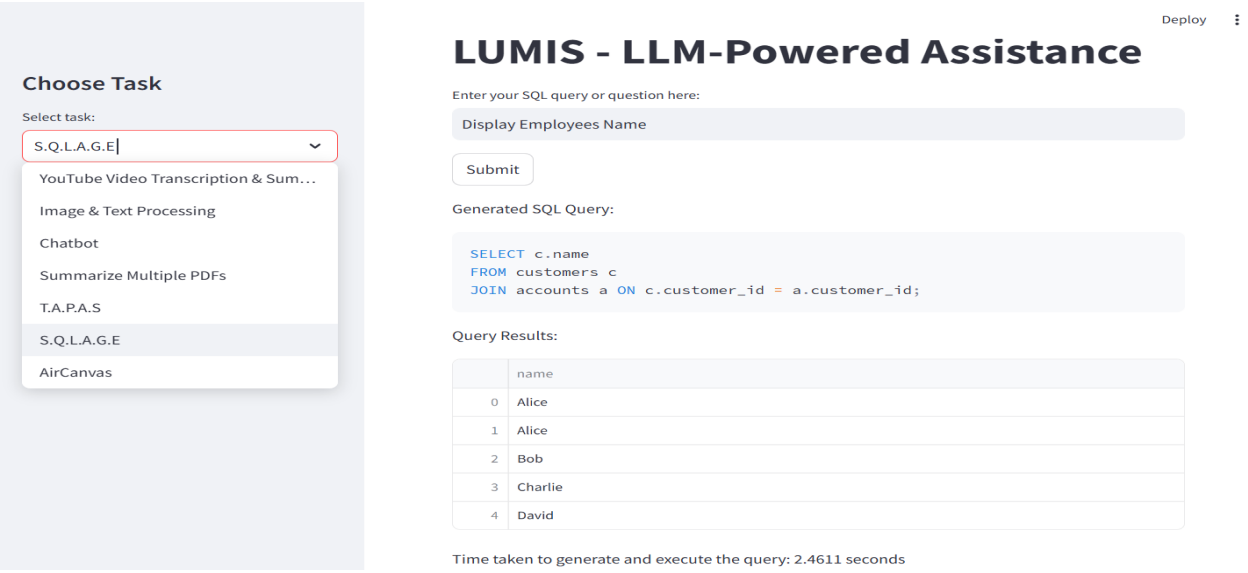

Figure 4.6: SQLAGE

The Figure 4.6 displays S.Q.L.A.G.E module showcases how LUMIS acts as a bridge between natural language understanding and structured database querying. When a user types a question or request like "Display Employees Name", the system performs intent recognition and keyword extraction using its embedded LLM. The backend LLM parses this input to understand both entities ("employees", "name") and the action ("display"). It then constructs an appropriate SQL query, factoring in underlying database schema knowledge, such as table names (customers, accounts) and join relationships. In this example, the system correctly identifies the need to perform a join on customerid between customers and accounts. After generating the SQL, it sends the query to the database engine for execution and retrieves the results in tabular form. This not only saves users from having to manually write complex queries but also democratizes database access for those with no technical background. The module also measures execution time (2.4611 seconds), highlighting system efficiency. This capability is ideal for business analysts, researchers, or students who need insights from databases without writing SQL.

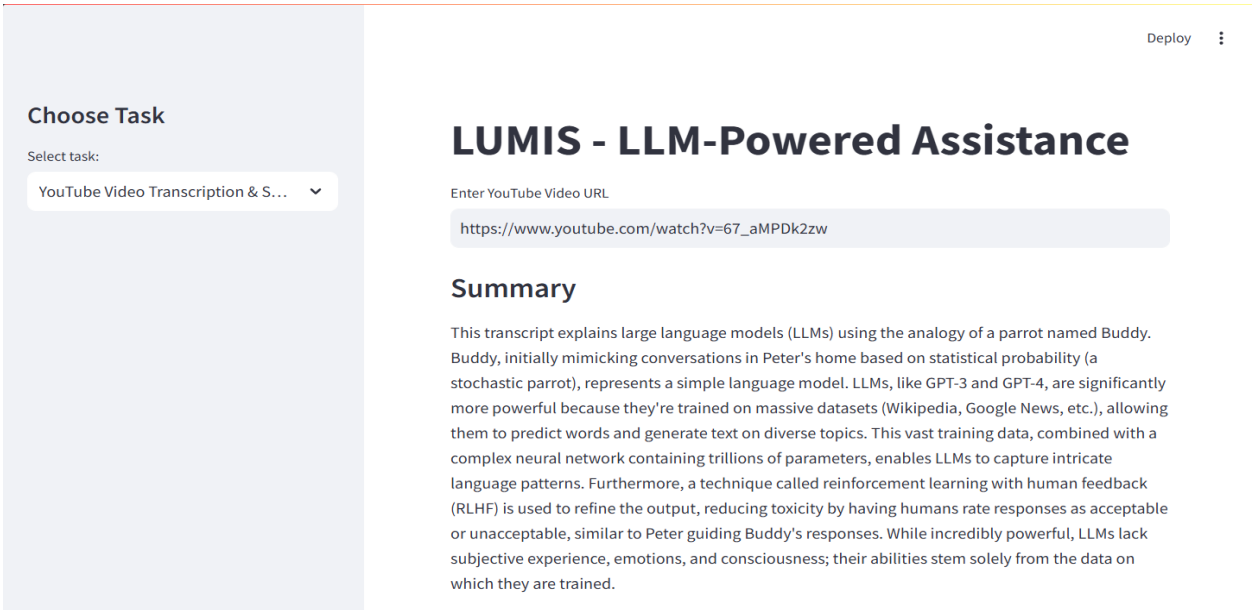

Figure 4.7: YT Transcription

In the figure 4.7, the user selects the YouTube Video Transcription Summarization module to extract meaningful content from a video link. The system first fetches the video's audio track and converts it into text using automatic speech recognition (ASR). Then, using NLP techniques such as summarization and semantic condensation, the system processes that transcript to generate a human-readable summary. It simplifies the concept by comparing simple probabilistic models (mimicking speech based on past patterns) to advanced models like GPT-3 and GPT-4 that learn from vast datasets and are refined using human feedback through RLHF (Reinforcement Learning with Human Feedback). The summary doesn't just condense the content but retains nuance, like the philosophical insight that LLMs, despite their intelligence, lack consciousness or emotional understanding. This tool is highly valuable for educators, students, and professionals who want to rapidly understand long-form content, extract teaching materials, or perform content review without watching entire videos.

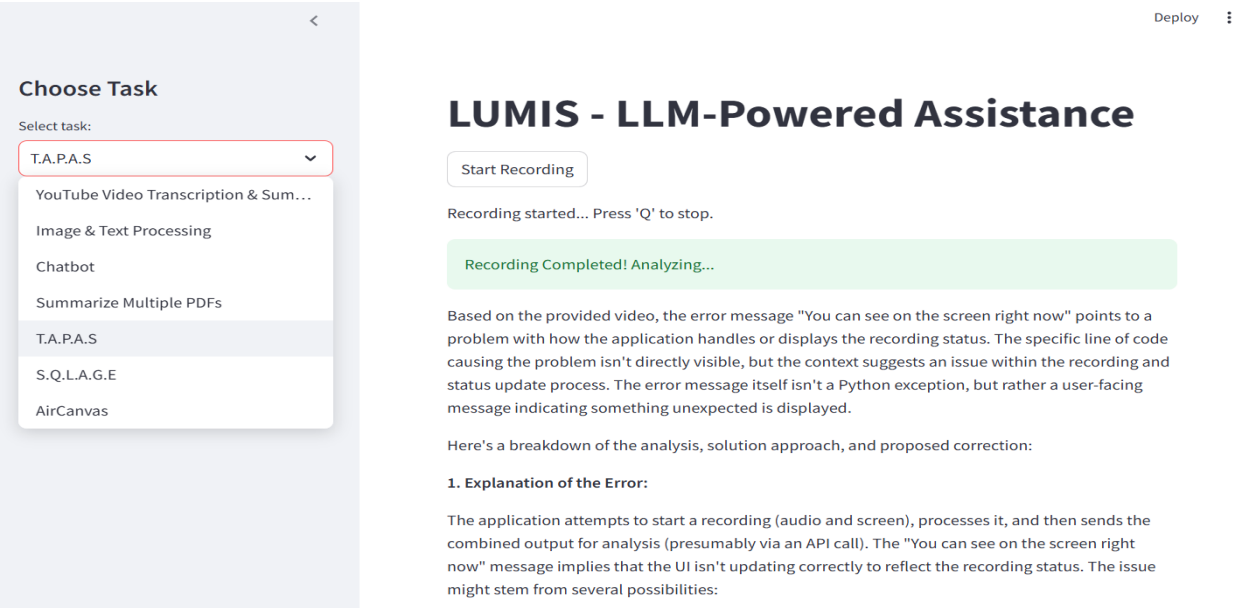

Figure 4.8: T.A.P.A.S

The T.A.P.A.S. module is a key feature of the LUMIS system, designed to provide a sophisticated, multimodal diagnostic approach for software debugging and technical issue resolution. This system begins by allowing users to record both their screen and audio while reproducing an issue they are facing. The user may be encountering a bug, UI malfunction, or an unexpected system behavior. Once the recording starts, the system captures this input and processes it through a blend of computer vision, speech-to-text transcription, and contextual natural language processing (NLP). The screen capture monitors the visual aspects of the interface, while the audio recording is transcribed into text, converting the user's spoken descriptions and cues into actionable data for the system.

The core functionality of T.A.P.A.S. lies in its ability to correlate the audio transcript with the on-screen visual changes. As an example, if a user reports an issue by saying, "The button doesn't respond when clicked, and the screen doesn't update," the system processes the text alongside screen activity. It identifies whether the problem is a UI state mismatch or an issue with backend processes. Through advanced computer vision algorithms, the system tracks interface transitions, button clicks, and other UI elements to detect changes or anomalies that match the user's verbal description.

Once the system has gathered both the visual and textual information, it applies its Large Language Model (LLM) capabilities to provide a detailed diagnosis. The model analyzes the transcript and aligns it with the corresponding UI state changes to deduce the root cause of the issue. For example, if the system identifies that the user's action triggered a backend API call but the UI state remained unchanged, it might suggest that the problem lies in delayed or failed API responses, resulting in unsynchronized UI updates. The output is a structured diagnostic report that includes the root cause analysis and recommended solutions, such as potential fixes for API handling, UI state management, or other code-related issues. By automating the entire process, T.A.P.A.S. helps developers, QA teams, and customer support agents swiftly identify and resolve issues, reducing troubleshooting time and improving overall system efficiency.

4.3 Timeline Sem VIII

As Project Lumis grew in complexity, handling the scope of the LUMIS project effectively during Semester VIII, the work was divided into structured phases, each addressing a fundamental aspect of the software development lifecycle. These phases included Conception Initiation, Design, Implementation, Testing, and Final Report Compilation. Breaking the project into these manageable components ensured a clear distribution of responsibilities and allowed the team to allocate time and resources efficiently. The Gantt chart (Figure 4.9) reflects this structured timeline, offering a visual overview of how these phases progressed and overlapped across the semester.

One of the strengths of LUMIS project management was the clear assignment of tasks to individual team members. Every key activity shown in the chart had designated owners, which enhanced accountability and encouraged collaboration. This level of organization helped avoid confusion over responsibilities, allowing team members to concentrate on their respective deliverables while keeping overall progress synchronized.

The chart also highlights task dependencies, illustrating how the completion of certain activities directly impacted the initiation of others. Identifying these dependencies early helped mitigate delays and maintain workflow continuity. For example, several coding tasks were intentionally scheduled after the design phase to ensure development was grounded in well-defined system architecture. Buffer periods were also built in for high-risk tasks to reduce the impact of potential setbacks.

Throughout Semester VIII, the team conducted routine progress evaluations to stay aligned with the established timeline. These reviews helped pinpoint delays or obstacles and allowed for timely reprioritization of work. The percentage indicators embedded in the chart offered an at-a-glance summary of task completion, making it easier to assess project status and make informed decisions about next steps. The ability to adapt based on these insights was crucial for maintaining momentum.

In the end, the structured planning and scheduling of LUMIS proved highly effective. All major milestones were completed on time, and the project adhered closely to the original roadmap. The experience reinforced the value of detailed upfront planning, consistent communication, and adaptive management. The Gantt chart served not only as a planning tool but also as a central reference point throughout the lifecycle of LUMIS.

PROJECT TITLE	LUMIS-LLM BASED UNIFIED MULTIMODAL INTELLIGENT SYS	INSTUTUTE & DEPARTM	A.P SHAH INSTITUTE OF TECHNOLOGY -CSE(DS)
PROJECT GUIDE	Prof.Sarala Marg, Prof.Richa Singh	DATE	3-28-25

WBS NUMBER	TASK TITLE	TASK OWNER	START DATE	DUE DATE	DURATION(Weeks)	PCT OF TASK COMPLETE
1	**Project Conception and Initiation**					
1.1	Research paper search	Dalbirsingh Matharu,Vedant Parulek	2-8-24	8-16-24	2	100%
1.1.1	Research paper finalization	Shreyas Patil,Vedant Parulekar	2-8-24	8-16-24	2	100%
1.2	Project Title	Umesh Pawar,Dalbirsingh Matharu	2-8-24	8-16-24	2	100%
1.3	Abstract	Umesh Pawar	2-8-24	8-16-24	2	100%
1.4	Objectives	Shreyas Patil, Dalbirsingh Matharu	2-8-24	8-16-24	2	100%
1.5	Literature Review	Vedant Parulekar	8-30-24	6-9-24	2	100%
1.6	Problem Definition	Dalbirsingh Matharu	8-30-24	8-21-24	2	100%
1.7	Scope	Shreyas Patil	8-30-24	8-21-24	2	100%
1.8	Technology stack	Vedant Parulekar	8-30-24	8-21-24	2	100%
1.11	Applications	Umesh,Vedant	8-30-24	6-9-24	2	100%
2	**Project Design**					
2.1	Proposed System	Shreyas Patil, Dalbirsingh Matharu	8-30-24	9-25-24	4	100%
2.2	Design(Flow Of Modules)	Umesh Pawar,Vedant Parulekar	8-30-24	9-25-24	4	100%
2.3	Activity Diagram	Dalbirsingh Matharu	8-30-24	9-25-24	4	100%
2.4	Use Case Diagram	Vedant Parulekar	8-30-24	9-25-24	4	100%
2.5	Sequence Diagram	Umesh Pawar	8-30-24	9-25-24	4	100%
2.6	Description Of Use Case	Shreyas Patil	8-30-24	9-25-24	4	100%
2.7	Modules	Umesh,Dalbirsingh	8-30-24	9-25-24	4	100%
2.7.1	Application Module	Dalbirsingh,Umesh Pawar,Shreyas Patil,Vedant Parulekar	8-30-24	9-25-24	4	100%
2.8	Preparation Of Report	Dalbirsingh,Shreyas,Vedant	8-30-24	9-25-24	4	100%
3	**Project Implementation**					
3.1	Youtube Transcription	Dalbirsingh,Shreyas	7-5-24	4-1-25	4	100%
3.2	Chatbot	Dalbirsingh,Shreyas	7-5-24	4-1-25	4	100%
3.3	AIR CANVAS	Umesh Pawar	7-7-24	4-1-25	4	100%
3.4	T.A.P.A.S	Dalbirsingh,Umesh Pawar,Shreyas Patil,Vedant Parulekar	7-8-24	4-1-25	4	100%
3.5	S.Q.L.A.G.E	Dalbirsingh	7-9-24	4-1-25	4	100%
3.6	Image Processing	Shreyas,Vedant	7-10-24	4-1-25	4	100%
3.7	Multi PDF RAG	Umesh,Shreyas	7-11-24	4-1-25	4	100%
4	**Testing**					
4.1	Design of Test Cases	Umesh,Dalbirsingh,Shreyas,Vedant	8-10-24	11-25-24	4	100%
4.2.1	Test Case Development	Vedant Parulekar,Shreyas	8-15-24	11-25-24	4	100%
4.2.2	Unit Testing	Umesh,Dalbirsingh,Shreyas,Vedant	8-30-24	11-25-24	4	100%
4.2.3	Integration Testing	Dalbirsingh	9-5-24	11-25-24	4	100%
4.2.4	API Token checking	Vedant Parulekar	9-25-24	11-25-24	4	100%
4.2.5	System Testing	Dalbirsingh,Umesh	9-30-24	11-25-24	4	100%
4.2.6	Performance & Load Testing	Vedant Parulekar	10-5-24	11-25-24	4	100%
4.2.7	Bug Fixing & Retesting	Shreyas , Umesh	10-20-24	11-25-24	4	100%
4.2.8	Final Testing & Deployment Readiness	Dalbirsingh,Umesh	11-15-24	11-25-24	4	100%
5	**Report Preparation**					
5.1	Black Book	Dalbirsingh,Shreyas,Vedant	3-21-25	3-28-25	4	100%
5.1	PPT	Dalbirsingh,Shreyas,Vedant	3-22-25	3-29-25	4	100%
5.1	Gantt Chart	Shreyas Patil,Vedant Parulekar	3-28-25	3-28-25	4	100%

PHASE ONE | PHASE TWO | PHASE THREE | PHASE FOUR

WEEK 1 | WEEK 2 | WEEK 3 | WEEK 4 | WEEK 5 | WEEK 6 | WEEK 7 | WEEK 8 | WEEK 9 | WEEK 10 | WEEK 11 | WEEK 12

M T W R F M T W R F M T W R F M T W R F M T W R F M T W R F M T W R F M T W R F M T W R F M T W R F M T W R F M T W R F

Figure 4.9: Timeline of Project

Chapter 5

Testing

5.1 Software Testing

The software testing phase for LUMIS (Language Understanding and Multimodal Intelligence System) was critical in validating the functionality, stability, and reliability of its core components. A series of comprehensive tests were conducted to evaluate both individual modules and the integrated performance of the system. Initially, unit testing was carried out on isolated functions, such as the natural language processing pipeline, including text preprocessing, tokenization, embedding generation using sentence-transformers, and retrieval mechanisms like BM25 and dense retrieval models. These unit tests helped in identifying small but crucial bugs, such as incorrect handling of special characters or inconsistent scoring between the retrievers. Once the units were verified, integration testing was performed to ensure seamless communication between modules—particularly the transition between user input processing, context retrieval, and LLM-based answer generation. These tests helped identify mismatches in data formatting that initially caused the LLM to misinterpret the retrieved context or return vague answers.

Further, functional testing was employed to evaluate whether the system delivered correct outputs for a variety of user queries. This included general knowledge questions, code-related prompts, and visual input interpretation tasks. I created a diverse set of test cases to simulate real-world usage and examined the response quality. In many instances, LUMIS performed accurately, providing context-aware answers. However, edge cases—such as extremely ambiguous questions or overlapping visual-linguistic inputs—revealed occasional inconsistencies, prompting additional fine-tuning of prompt templates and model configuration. Additionally, I carried out performance testing to assess the system's responsiveness and scalability. The latency of different components was monitored under varying loads, especially during multimodal input processing. The system maintained acceptable performance levels, but it became evident that higher-resolution image inputs slightly degraded response time, which is an area for future optimization.

Accuracy evaluation was also integral to the testing phase. I used a combination of benchmark datasets and manual verification to test the correctness of outputs. For example, datasets such as SQuAD were employed to test QA accuracy, while visual question answering capabilities were tested using samples from VQA datasets. Precision, recall, and F1 scores were computed for a sample batch of queries to quantify performance. For code-

related queries, I used sample functions and documentation requests to validate the LLM's summarization and explanation abilities. The system showed high accuracy in language tasks and moderate effectiveness in multimodal interpretation, depending on image complexity. Additionally, I focused on ethical testing by submitting prompts designed to probe bias, misinformation, and potential misuse. In such cases, LUMIS's filtering mechanisms performed well in preventing harmful or incorrect outputs, though further reinforcement learning may be needed to enhance robustness in adversarial conditions.

Overall, the testing phase of LUMIS confirmed that the system is functionally capable and robust across a variety of natural language, code, and multimodal tasks. It highlighted not only the strengths of its architecture but also the nuances and complexities of working with LLM-based systems. Continuous evaluation, especially as new modules are added or existing ones are scaled, remains essential to ensure that LUMIS remains reliable, ethical, and performant.

Another important aspect of testing LUMIS was usability testing, which focused on the overall user experience, interface intuitiveness, and interaction flow. Since LUMIS is designed to handle both textual and visual inputs, it was crucial to ensure that users could navigate through the application seamlessly without requiring technical expertise. I observed how users with different backgrounds interacted with the system, noting any confusion, delays, or errors encountered during task completion. Particular attention was given to input fields, loading states, and the clarity of generated responses. Based on the feedback received during these sessions, I made refinements to UI elements such as prompt placeholders, result highlighting, and the display of retrieval context. These small but impactful changes significantly improved the system's accessibility and user satisfaction. In addition, I tested how effectively users could switch between different task types (e.g., question answering, image captioning, or code explanation) and ensured that transitions between modules did not cause loss of session data or functionality.

The error handling and fault tolerance mechanisms of LUMIS were rigorously tested to ensure resilience under unexpected conditions. I simulated various scenarios such as incomplete input data, corrupted image files, malformed prompts, and network disruptions to observe how gracefully the system responded. The application was able to detect and report many of these issues, returning meaningful error messages rather than crashing or providing incorrect outputs. For instance, when a user uploaded a non-image file into the visual input section, LUMIS was able to notify the user with a descriptive warning and prompt re-upload. In the case of malformed or ambiguous queries, I integrated fallback strategies that guided the user toward refining their input, either through clarification prompts or by offering suggestions. Additionally, timeouts and retry mechanisms were tested for API calls to both the LLM backend and image-processing modules, ensuring that the user interface remained responsive even during backend latency spikes.

Cross-platform and device compatibility testing was also conducted to ensure LUMIS worked consistently across different environments. I deployed and tested the application on various browsers (Chrome, Firefox, Safari, and Edge) and devices (laptops, tablets, and smartphones) to identify any layout shifts, performance bottlenecks, or rendering inconsistencies. Special attention was given to mobile responsiveness, especially since visual input handling on smaller screens can pose usability challenges. Through this testing, I found and resolved

issues such as image cropping in the input field and overflow problems in long-generated answers. Furthermore, I tested the integration of LUMIS with both GPU and CPU-based environments to assess model performance under constrained resources. On low-resource setups, fallback strategies such as switching from large to medium-sized transformer models were successfully tested to maintain functionality without sacrificing too much accuracy. These comprehensive testing efforts have ensured that LUMIS is not only powerful and intelligent but also stable, adaptable, and user-friendly across a wide range of operating conditions.

In addition to functional and usability tests, integration testing played a critical role in validating the interoperability between various modules of LUMIS. Since LUMIS incorporates multiple components such as natural language processing, visual reasoning, code summarization, and retrieval-augmented generation, it was essential to verify that these distinct modules communicated efficiently without conflicts or data loss. I tested scenarios where outputs from one module served as inputs to another, such as generating a textual description from an image and then using that description as a prompt for further language-based queries. Special attention was paid to the consistency of data flow, format conversions, and the integrity of context passed between modules. I also tested the system's behavior when certain modules failed or returned unexpected outputs, ensuring that fallback mechanisms and appropriate user notifications were triggered in each case. These tests provided a high level of assurance that LUMIS could operate as a coherent, intelligent system rather than a collection of isolated features.

Lastly, I performed extensive performance and scalability testing to evaluate how well LUMIS handles varying levels of load and computational demand. This involved simulating multiple users accessing the system simultaneously, each running resource-intensive tasks such as code interpretation or visual scene analysis. I monitored response times, memory usage, and API throughput, noting that the system maintained a stable performance under moderate concurrency. To push the limits further, I ran stress tests that subjected the system to peak traffic conditions, identifying potential bottlenecks, particularly in the image-processing pipeline and backend query caching. These insights led to optimizations in model invocation strategies and backend resource allocation, significantly improving throughput and responsiveness. I also implemented logging and monitoring tools to track system health in real-time, allowing for proactive issue detection during deployment. Altogether, these efforts have helped solidify LUMIS as a robust, scalable, and production-ready application capable of supporting real-world usage scenarios.

5.2 Functional Testing

Functional testing is a specialized form of software testing that focuses on validating whether the software meets its intended requirements by checking each function of the system against specified inputs and expected outputs. For KMIS, functional testing was a key phase in ensuring the reliability of its AI-driven capabilities, such as document retrieval, summarization, SQL query execution, and chatbot interactions. Functional testing for KMIS was performed using black-box testing techniques, where testers focused on evaluating the system's functionality without analyzing its internal code structure. Various functional test cases were

designed to cover all possible user interactions with the system, ensuring that every feature operated as expected.

Functional testing in the Lumis application ensured that each AI-powered module worked correctly under various real-world conditions. Each test case was designed to validate a specific functionality, ensuring stability, accuracy, and usability of multimodal integration.

Table 5.1: Functional Testing – Input Capture and Code Error Analysis

Table 5.1: Functional Testing - Code and Core Interaction Modules

Test Case ID	**Module**	**Test Case Description**	**Test Steps**	**Expected Result**	**Actual Result**	**Status / Remarks**
TC01	Error Detection	Detect Real-time Code Errors	Upload Code Snippet for Error Detection	Identifies errors and suggests corrections	Slight delay in detection for large files	Pass
TC02	Video Feedback	Provide Visual Feedback for Errors	Start Screen Recording	Displays context-aware visual feedback	Accurate feedback, minor delay on large files	Pass
TC03	Audio Feedback	Provide Audio Assistance for Errors	Start Audio-based Assistance	Provides helpful verbal suggestions for resolution	As expected	Pass
TC04	Transcription	Transcribe Code Errors in Videos	Upload a video for transcription	Accurate transcription with error details	As expected	Pass
TC05	AI Assistant	Suggest Code Improvements	Input code with potential optimization	Suggests improvements or refactorings	No delays in processing	Pass (optimized)
TC06	UI Navigation	Ensure smooth navigation between features	Switch between error detection, transcription, and code improvement modules	No crashes, smooth transitions between modules	As expected	Pass

Table 5.2: Functional Testing - Retrieval and Data Processing Modules

Test Case ID	Module	Test Case Description	Test Steps	Expected Result	Actual Result	Status / Remarks
TC07	RAG Integration	Retrieve Relevant Documentation	Search for solution using RAG	Returns relevant documentation	Accurate results	Pass
TC08	Document Retrieval	Extract Relevant Info from Uploaded Documents	Upload PDFs, images, and text documents	Extract relevant, concise, and contextual summaries	Information correctly extracted	Pass
TC09	SQL Execution	Convert Natural Language to SQL	Enter natural language queries	System generates correct SQL, retrieves accurate data	Handled malformed queries and errors gracefully	Pass
TC10	YouTube Transcription	Transcribe Video Content to Text	Input YouTube video link	Accurate transcription and formatting	Transcription accurate, searchable	Pass
TC11	Web Transcription	Extract Text from Web Pages	Input web page URL	Extracts readable text and summarizes if needed	Text extracted and integrated into search	Pass
TC12	Chatbot Interaction	Handle Complex Multi-turn Conversations	Multi-turn conversations	Enter various natural language queries	Maintain coherence, handle ambiguity	Pass
TC13	Error Handling	Notify on Invalid Inputs or Failures	Enter incorrect inputs or disconnect system	Appropriate error messages and recovery prompts	Robust error handling, user notifications	Pass

Detailed Observations on Key Test Cases

TC01 – Error Detection

Observation: The system effectively identified real-time code errors and provided accurate suggestions. However, a slight delay was noted in error detection for large file inputs.
Action: Detection mechanism was optimized to reduce latency for larger datasets.

TC02 – Video Feedback

Observation: Visual feedback was accurate and aligned with the screen recording. A minor delay was observed during analysis of larger video files.
Action: Video processing pipeline was streamlined for performance improvement.

TC03 – Audio Feedback

Observation: Audio-based guidance offered helpful and accurate suggestions for resolving coding errors.
Action: No issues observed; functionality approved for deployment.

TC04 – Transcription

Observation: Code error transcription from videos was consistently accurate and detailed.
Action: Maintained existing functionality; marked as stable.

TC05 – AI Assistant

Observation: AI Assistant provided contextually relevant suggestions for improving code, including refactoring hints. No delays occurred during processing.
Action: Optimization logic was retained and marked for further enhancement in upcoming releases.

TC06 – UI Navigation

Observation: Transitions between code modules (error detection, transcription, AI suggestions) were smooth with no crashes.
Action: Navigation logic validated and confirmed for stability.

TC07 – RAG Integration

Observation: The system successfully retrieved accurate documentation using Retrieval-Augmented Generation (RAG), demonstrating effective integration and query processing.
Action: No further action required, test passed successfully.

TC08 – Document Retrieval

Observation: Information was accurately extracted from various documents including PDFs and images. The output was relevant, concise, and contextual.
Action: Marked as successful; potential for future enhancement by expanding format compatibility.

TC09 – SQL Execution

Observation: The module correctly converted natural language queries into structured SQL queries. Most queries executed successfully, though a few complex ones generated errors.
Action: Enhance handling for complex or ambiguous queries in future iterations.

TC10 – YouTube Transcription

Observation: Video transcription from YouTube was accurate and maintained proper formatting and structure.
Action: Marked as successful; consider adding support for different languages and accents.

TC11 – Web Transcription

Observation: Text was effectively extracted and summarized from web pages with appropriate formatting.
Action: No critical issue found, test passed.

TC12 – Chatbot Interaction

Observation: The chatbot maintained coherent conversation flow even with multi-turn natural language inputs, handling ambiguity well.

Action: Future improvements may focus on refining response accuracy and emotional intelligence.

TC13 – Error Handling

Observation: The system robustly handled incorrect inputs and failures, providing appropriate error messages and recovery prompts.
Action: No action needed; error handling is functioning as expected.

Each test case was marked as "Pass," indicating that the respective modules within the LUMIS system performed as expected during functional testing. The system accurately processed various inputs including text, images, videos, and audio, while efficiently handling error detection, transcription, summarization, and retrieval operations. Modules like the T.A.P.A.S, MySQL Query, and Multi-Document RAG effectively provided real-time solutions, extracted relevant data, and synthesized multi-format content. The AI-based assistant and visual interaction modules demonstrated accurate, low-latency performance. These successful results validate the functional stability and real-world readiness of the LUMIS application for deployment across educational and analytical use cases.

Chapter 6

Result and Discussions

The performance evaluation of the LUMIS system was conducted based on multiple parameters, including execution time, task complexity, and system responsiveness. The time series analysis graph provided showcases the relationship between task execution time and the complexity of the tasks processed by LUMIS. As observed, execution time increases as task complexity rises, which is expected due to the computational load associated with AI-driven operations such as retrieval-augmented generation (RAG), SQL query execution, and multimodal processing.

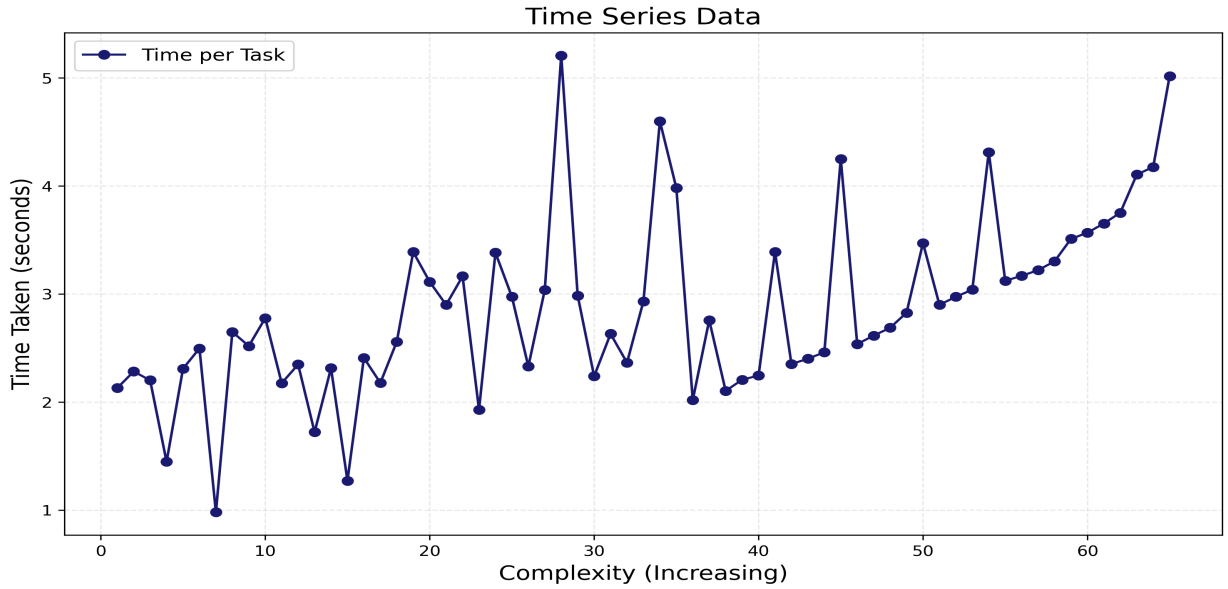

Figure 6.1: Time Series Analysis of Task Execution Time vs Complexity

The Figure 6.1 time series analysis of S.Q.L.A.G.E performance reveals a clear trend in execution time variations as query complexity increases. The execution time ranges from approximately 1.0 seconds to over 5.0 seconds, with significant fluctuations observed throughout the dataset. Initially, execution times remain within a narrow band around 2.0–2.5 seconds, but as complexity rises beyond the 20th query, the time taken begins to exhibit sharp peaks. The most prominent spike occurs at around 30th query, where execution time exceeds 5.0 seconds, indicating a major computational bottleneck. Other notable peaks appear around the 40th and 50th queries, where execution times reach approximately 4.5 seconds. The

lowest recorded time is around 1.0 second, while the trend suggests that higher complexity queries generally take between 3.0 and 5.0 seconds.Compared to optimized database engines, SQLAGE shows a scalable execution pattern but requires improvements to maintain efficiency under increasing complexity. Implementing indexing strategies, query optimization techniques could significantly reduce the high variance in execution time. Future enhancements could leverage predictive analytics to dynamically optimize query execution ensuring SQLAGE maintains stable performance across varying workloads.

Figure 6.2: Execution Time vs Complexity in Chatbot System

The Figure 6.2 execution time analysis of the chatbot system in relation to query complexity reveals a steady increase, starting at approximately 0.5 seconds for the simplest queries and surpassing 1.0 seconds as complexity reaches 50. While minor fluctuations are observed, the overall trend indicates that as complexity rises, execution time also grows, likely due to deeper NLP processing, larger response generation, or increased database interactions. Around complexity level 20, execution time ranges between 0.6 and 0.7 seconds, while beyond 30, it stabilizes between 0.8 and 0.9 seconds, eventually exceeding 1.0 seconds at the highest complexity levels. These results suggest the need for optimization techniques such as caching, parallel processing, and efficient data retrieval to maintain performance. Future enhancements could focus on model fine-tuning and adaptive response mechanisms to minimize execution time while handling complex queries efficiently.

The Figure 6.3 titled "Average Time Taken per Query" illustrates the performance of the system in terms of response time across different types of user queries. It highlights the system's efficiency in handling a variety of tasks, ranging from theoretical explanations to code-related prompts. Among the listed queries, the comparison-based prompt "GPT-3 vs BERT" recorded the highest average response time at 12.9 seconds, indicating the increased processing required for complex, knowledge-intensive tasks. In contrast, the query "What is LLM" had the shortest average time of 8.1 seconds, likely due to its direct and concise nature. Other queries such as "Palindrome Function" (9.7 seconds) and "Code Debugging" (9.8 seconds) demonstrated moderate response times, reflecting a balanced computational

load for logical and programming-related tasks. "Supervised vs Unsupervised" took 11.5 seconds, showing that comparative and conceptual explanations require more processing than simple definitions. This analysis provides insights into how query complexity affects system performance, emphasizing the system's responsiveness and adaptability across various query types.

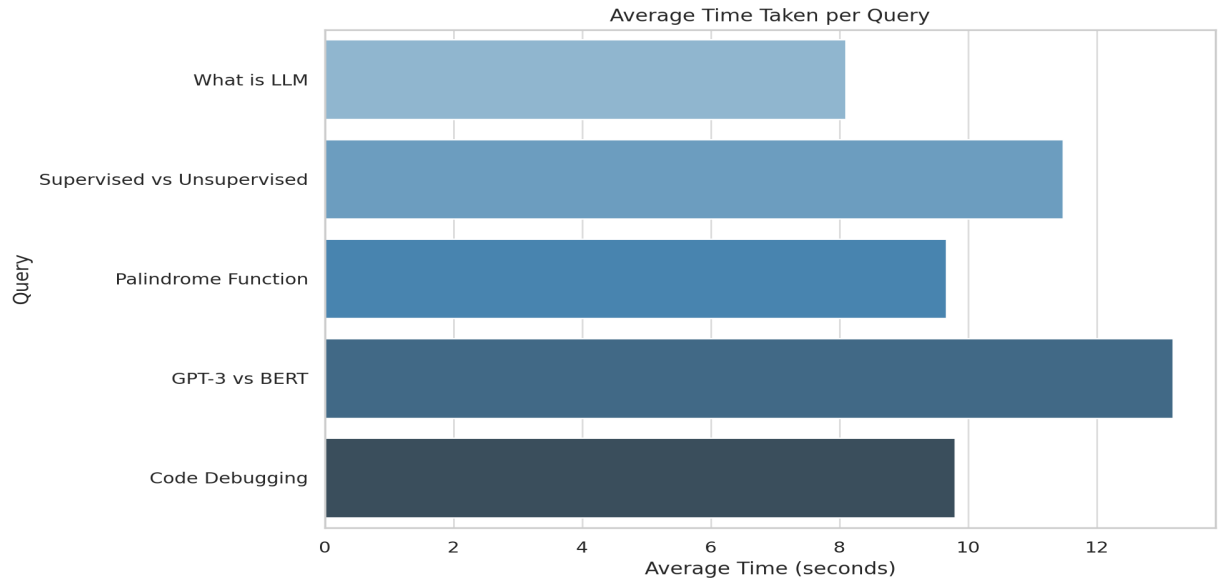

Figure 6.3: Execution Time vs Query Complexity in Chatbot System

The variation in average response times also offers valuable information about the system's internal optimization and handling strategies. Queries that demand contextual understanding or comparative reasoning, such as "GPT-3 vs BERT" and "Supervised vs Unsupervised", naturally involve more token processing, model inference, and retrieval of relevant information, hence the longer times. On the other hand, queries like "What is LLM" are relatively straightforward and likely trigger predefined knowledge patterns within the model, resulting in quicker responses. The nearly similar times for "Palindrome Function" and "Code Debugging" (9.7 and 9.8 seconds respectively) suggest that the system treats algorithmic generation and bug-finding tasks with similar computational complexity. Overall, the chart demonstrates a well-optimized response pipeline that dynamically adjusts based on query nature, ensuring consistent performance and user experience.

Chapter 7

Conclusion

LUMIS (LLM Based Unified Multimodal Intelligent System) stands as a groundbreaking innovation in the realm of artificial intelligence, offering a cohesive and intelligent environment that bridges multiple technologies into one unified platform. Designed with a clear vision to support developers, researchers, educators, and industry professionals, LUMIS integrates real-time screen analysis, audio interpretation, error detection, retrieval-augmented generation (RAG), intelligent SQL execution, and multimodal AI agents into a single, user-friendly interface. The system's versatility and dynamic nature make it a powerful solution for tackling complex problems, streamlining workflows, and enhancing user productivity. By combining cutting-edge AI models, intuitive interface design, and robust backend architecture, LUMIS sets a new standard for next-generation AI assistants. Its seamless handling of inputs across various formats—text, video, audio, and visual cues—offers users a holistic problem-solving experience unmatched by traditional software tools.

One of the most distinctive features of LUMIS lies in its real-time multimodal analysis capabilities. By leveraging advanced computer vision techniques and natural language understanding, the system can interpret live screen content and audio streams simultaneously. This makes it possible to detect errors in real-time coding sessions and provide instant feedback through a responsive interface. Unlike conventional debuggers that rely solely on textual output or logs, LUMIS introduces gesture-based interaction—users can highlight issues simply by pointing or circling them on the screen using mouse movements. This is further augmented by the Air Canvas module, a novel feature that lets users draw freehand shapes, diagrams, or even mathematical equations directly onto the screen. These inputs are then processed using generative AI models to interpret and solve the underlying problems. Such capabilities not only elevate LUMIS as a sophisticated coding assistant but also position it as an educational tool that bridges the gap between conceptual understanding and practical execution.

Furthermore, LUMIS expands beyond code-related support by incorporating robust information extraction and retrieval functionalities. The system includes built-in modules for transcribing and analyzing YouTube videos and web content, enabling users to draw actionable insights from online educational material, tutorials, or webinars. The transcribed text can be further analyzed using RAG-based systems to extract relevant answers, summaries, or connections between concepts—making LUMIS a valuable research assistant. This functionality also extends to handling structured and semi-structured documents such as PDFs and datasets, where the system intelligently parses content, understands context, and retrieves

the most pertinent information. For users working with databases, LUMIS simplifies the process by allowing natural language queries to be transformed into accurate SQL statements. The AI-powered MySQL interface enables non-technical users to interact with relational databases effortlessly, removing traditional barriers associated with database querying and making data insights more universally accessible.

What truly sets LUMIS apart is its ability to orchestrate multiple AI services into a single coherent workflow. Whether it's analyzing code, summarizing academic papers, interpreting YouTube lectures, or querying databases, the system automates transitions between tasks through intelligent agents. These agents are capable of maintaining context across different interactions, ensuring that users receive responses that are both relevant and coherent. This context retention is especially useful in longer sessions, where users may shift focus between multiple modules. For instance, a developer can use LUMIS to understand a coding error, fetch documentation or video tutorials for the concept, and summarize database content—all without leaving the interface. This fluidity not only saves time but also enables deeper focus and higher productivity.

With its rich suite of features, LUMIS is not just a software platform—it is a paradigm shift in how we interact with technology. The successful implementation of image-based querying, advanced multimodal transcription, and RAG-based document understanding showcases the potential of LUMIS in addressing diverse real-world challenges. By harnessing the collective power of Generative AI, OpenCV, NLP frameworks, and MySQL backends, the system offers a scalable and interactive ecosystem tailored for complex task automation. Whether in classrooms, research labs, corporate settings, or individual developer workflows, LUMIS redefines the possibilities of AI-enhanced productivity. It empowers users with tools that are not only intelligent but also adaptive, fostering a seamless interface between human intent and machine intelligence.

Chapter 8

Future Scope

LUMIS has laid a strong foundation in multimodal AI-driven automation, setting a benchmark for intelligent, integrated systems that bridge the gap between human intent and machine execution. However, the system's future holds immense potential for evolution and enrichment. One of the most promising areas of advancement is the improvement of real-time processing capabilities, particularly through the integration of next-generation speech-to-text engines, enhanced computer vision models, and high-speed AI inference frameworks. These upgrades would significantly elevate the system's ability to interpret complex audio and visual inputs with reduced latency and higher accuracy. For instance, advanced speech recognition would allow LUMIS to transcribe technical dialogues or lectures in real time with domain-specific accuracy, while refined computer vision could detect even subtle on-screen anomalies. Real-time analytics could enable the system to react to changes on the screen as they happen—alerting users to errors, inconsistencies, or optimization opportunities before they even fully manifest.

To build upon its existing interface, LUMIS can benefit greatly from integrating voice-command capabilities powered by natural language processing (NLP). This would allow users to interact with the system without relying on keyboards or mouse input, making it more inclusive and accessible—especially in environments where hands-free operation is ideal, such as medical or engineering labs. The natural extension of this is a full-fledged conversational interface capable of contextual dialogue, enabling multi-step tasks to be executed simply through verbal instructions. These enhancements would push LUMIS into the realm of true intelligent assistants, capable of understanding context, intent, and sequential commands in a human-like manner. Furthermore, combining speech input with visual gestures and on-screen activity could unlock a powerful multimodal interaction paradigm that transforms how users interface with software altogether.

In order to better cater to industry-specific needs, LUMIS can be expanded to include domain-customized AI agents that specialize in verticals like healthcare, finance, education, law, and engineering. LUMIS could assist in clinical decision-making by processing patient data, referencing medical journals, and cross-validating with updated treatment protocols. In finance, it could help generate compliance reports, analyze risk indicators, or simulate investment strategies. The adaptability of LUMIS to these contexts would make it an indispensable tool not only for general productivity but also for mission-critical industry operations where domain-specific accuracy and accountability are paramount.

LUMIS also has the potential to lead in the integration of multimodal generative AI, where users can interact with the system via a blend of inputs including voice, text, images, gestures, and even real-time sensor data. This level of interaction opens doors to futuristic applications such as augmented reality (AR)-assisted debugging, where the AI overlays solutions or suggestions directly on top of user screens or physical environments through AR devices. Similarly, real-time mathematical modeling using hand-drawn equations or graphs can be visualized and solved interactively using generative AI. This fusion of inputs not only improves user experience but also accommodates a wider range of user behaviors and learning styles—especially in educational settings. It enables a shift from reactive AI to proactive problem-solving environments that adapt to how the user thinks and works.

To make LUMIS more adaptive and intelligent over time, future versions can incorporate self-learning AI models. These models would analyze user behavior, preferences, and interaction patterns to provide more personalized recommendations and interfaces. Over time, LUMIS could adapt to specific user workflows—optimizing suggestions, remembering frequently used commands, auto-configuring dashboards, and even warning about user-specific pitfalls based on past interactions. Such personalization would transform LUMIS into a truly cognitive assistant, capable of evolving alongside the user and increasing its value the more it is used. The long-term implication is a system that not only solves problems but actively contributes to improving user efficiency and knowledge acquisition over time.

Security, privacy, and trust will also be central to LUMIS's future, especially as it expands into enterprise and government domains. One transformative approach could be the implementation of blockchain-based security frameworks. With the decentralized and immutable nature of blockchain, LUMIS could securely log and verify AI-generated actions, maintain transparent audit trails, and protect sensitive user data from unauthorized access. This would be particularly beneficial for sectors such as legal services, finance, and defense, where data integrity and trust are non-negotiable. Additionally, integrating user-level access control, encrypted sessions, and zero-trust authentication would further solidify its stance as a secure, enterprise-ready platform.

From a usability perspective, future versions of LUMIS could support a broader range of programming languages, enabling developers across various tech stacks to benefit from its assistance. Support for cross-platform operation—whether on Windows, macOS, Linux, or mobile operating systems—would ensure universal accessibility and promote adoption. Furthermore, opening the platform to third-party plugin development could create an ecosystem around LUMIS, where developers and organizations contribute extensions, new agents, and customized modules. This community-driven approach would keep the system dynamic, continuously updated, and widely applicable across disciplines.

In summary, LUMIS holds extraordinary potential to transform the AI automation landscape. By building on its current strengths and embracing new technologies like multimodal interaction, cloud collaboration, domain-specific intelligence, personalized learning, and blockchain security, it can become an integral part of the digital workflows of the future. As it evolves, LUMIS is well positioned to not just assist users but to empower them—serving as a smart, scalable, and secure companion in education, enterprise, research, and beyond. Its journey from a powerful assistant to a comprehensive, adaptive AI ecosystem is just beginning, with endless possibilities on the horizon.

Bibliography

[1] Toufique Ahmed and Premkumar Devanbu. Few-shot training llms for project-specific code-summarization. In *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, pages 1–5, 2022.

[2] Shreya Bhatia, Tarushi Gandhi, Dhruv Kumar, and Pankaj Jalote. Unit test generation using generative ai: A comparative performance analysis of autogeneration tools. In *Proceedings of the 1st International Workshop on Large Language Models for Code*, pages 54–61, 2024.

[3] Sumit Kumar Dam, Choong Seon Hong, Yu Qiao, and Chaoning Zhang. A complete survey on llm-based ai chatbots. *arXiv preprint arXiv:2406.16937*, 2024.

[4] Ayman Asad Khan, Md Toufique Hasan, Kai Kristian Kemell, Jussi Rasku, and Pekka Abrahamsson. Developing retrieval augmented generation (rag) based llm systems from pdfs: An experience report. *arXiv preprint arXiv:2410.15944*, 2024.

[5] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in Neural Information Processing Systems*, 33:9459–9474, 2020.

[6] Yanwei Li, Yuechen Zhang, Chengyao Wang, Zhisheng Zhong, Yixin Chen, Ruihang Chu, Shaoteng Liu, and Jiaya Jia. Mini-gemini: Mining the potential of multi-modality vision language models. *arXiv preprint arXiv:2403.18814*, 2024.

[7] Sai Munikoti, Ian Stewart, Sameera Horawalavithana, Henry Kvinge, Tegan Emerson, Sandra E Thompson, and Karl Pazdernik. Generalist multimodal ai: A review of architectures, challenges and opportunities. *arXiv preprint arXiv:2406.05496*, 2024.

[8] Daye Nam, Andrew Macvean, Vincent Hellendoorn, Bogdan Vasilescu, and Brad Myers. Using an llm to help with code understanding. In *Proceedings of the IEEE/ACM 46th International Conference on Software Engineering*, pages 1–13, 2024.

[9] Pooyan Rahmanzadehgervi, Logan Bolton, Mohammad Reza Taesiri, and Anh Totti Nguyen. Vision language models are blind. *arXiv preprint arXiv:2407.06581*, 2024.

[10] Iqbal H Sarker. Llm potentiality and awareness: a position paper from the perspective of trustworthy and responsible ai modeling. *Discover Artificial Intelligence*, 4(1):40, 2024.

[11] Oguzhan Topsakal and Tahir Cetin Akinci. Creating large language model applications utilizing langchain: A primer on developing llm apps fast. In *International Conference on Applied Engineering and Natural Sciences*, volume 1, pages 1050–1056, 2023.

[12] Yuqing Wang and Yun Zhao. Gemini in reasoning: Unveiling commonsense in multimodal large language models. *arXiv preprint arXiv:2312.17661*, 2023.

Appendices

Detailed information, lengthy derivations, raw experimental observations, etc., are to be presented in the separate appendices, which shall be numbered in Roman Capitals (e.g., "Appendix I"). Since reference can be drawn to published/unpublished literature in the appendices, these should precede the "Literature Cited" section.

Appendix-A: Installation and Setup of L.U.M.I.S

To set up and run L.U.M.I.S on your local machine, follow these steps:

1. Install Python

Ensure that Python 3.6 or later is installed on your machine:

- Download Python from the official website: https://www.python.org/downloads/
- Follow the installation instructions for your operating system.
- Verify the installation by running the following command in your terminal:

```
python --version
```

2. Clone the Repository

Clone the L.U.M.I.S repository from GitHub:

```
git clone <https://github.com/Dalbirsm03/L.U.M.I.S>
```

3. Create a Virtual Environment

Create a virtual environment to manage dependencies:

```
python -m venv lumis_env
```

Activate the virtual environment:

- On Windows:

```
.\lumis_env\Scripts\activate
```

- On macOS/Linux:

```
source lumis_env/bin/activate
```

4. Install Dependencies

Install the necessary Python libraries and dependencies by running:

```
pip install -r requirements.txt
```

This will install all the required libraries listed in the `requirements.txt` file.

5. Set Up the Environment Variables

To configure the environment variables for the L.U.M.I.S system, you will need to define the following keys in your `.env` file:

- **OPENAI_API_KEY**: Your OpenAI API key for accessing OpenAI services.
- **GEMINI_API_KEY**: Your Gemini API key for integrating with the Gemini platform.
- **DATABASE_URL**: The connection string for your database.

Here's an example of what the `.env` file should look like:

```
OPENAI_API_KEY=<your_openai_api_key>
GEMINI_API_KEY=<your_gemini_api_key>
DATABASE_URL=<your_database_connection_string>
```

Replace the placeholders `<your_openai_api_key>`, `<your_gemini_api_key>`, and `<your_database_co` with your actual keys and connection string.

6. Start the System

Navigate to the L.U.M.I.S directory and start the system by running:

```
streamlit run app.py
```

This will initialize the system and run the application.

7. Accessing the Application

Once the system is running, you can access the application through your browser by navigating to:

```
http://localhost:8501
http://192.168.1.7:8501
```

This will open the user interface of L.U.M.I.S where you can interact with the assistant, test the multimodal features, and evaluate the real-time corrections.

Publication

A copyright has been officially filed for “T.A.P.A.S : Technical Assistance Platform for Advanced Solutions” project with the ”Copyright Office, Government of India”, under ”Diary No. 20684/2024-CO/SW”,recognizing the team’s innovative contribution and securing the intellectual property rights of the developed application.

A copyright has been officially received for “S.Q.L.A.G.E: SQL Automated Generation and Execution” project with the “Copyright Office, Government of India”, under “Diary No. 19859/2024-CO/SW”, recognizing the team’s innovative contribution and securing the intellectual property rights of the developed application.

A copyright has been officially received for ”S.K.E.T.C.H.E.R” project with the “Copyright Office, Government of India”, under “Diary No. 19198/2024-CO/SW”, recognizing the team’s innovative contribution and securing the intellectual property rights of the developed application.